

FOREST WILD FIRE AND SMOKE DETECTION USING DEEP LEARNING

A Capstone Project Report submitted to
MOHAN BABU UNIVERSITY
In partial fulfilment of the requirements for the award of the
b-tech

**B-TECH OF COMPUTER SCIENCES
IN
DEPARTMENT OF COMPUTER SCIENCE**

Submitted by

KURUVA RAKSHITHA

23102A010433

Under the Esteemed Supervision of
Dr. K. MUNISANKAR
Assistant Professor



Department of Computer Science (AIML)
School of computing
MOHAN BABU UNIVERSITY
Sree Sainath Nagar, Tirupati - 517102

2025-26



MOHAN BABU UNIVERSITY

Vision

To be a globally respected institution with an innovative and entrepreneurial culture that offers transformative education to advance sustainability and societal good.

Mission

- ❖ Develop industry-focused professionals with a global perspective.
- ❖ Offer academic programs that provide transformative learning experience founded on the spirit of curiosity, innovation, and integrity.
- ❖ Create confluence of research, innovation, and ideation to bring about sustainable and socially relevant enterprises.
- ❖ Uphold high standards of professional ethics leading to harmonious relationship with environment and society.

SCHOOL OF COMPUTING

Vision

To be a center of excellence in computing education, research, and innovation by developing skilled professionals and responsible technologists for global societal needs.

Mission

- ❖ Provide quality education in Computer Science and emerging technologies through industry-oriented curriculum and practical learning.
- ❖ Encourage research, innovation, and entrepreneurship in the fields of Artificial Intelligence, Machine Learning, Data Science, and Cyber Security.
- ❖ Develop technical, analytical, and problem-solving skills to meet global industry standards.
- ❖ Promote ethical values, teamwork, leadership qualities, and lifelong learning among students.

DEPARTMENT OF COMPUTER SCIENCE

Vision

To become a Nation's center of excellence in the field Computer Sciences and its allied areas through teaching, training, and research.

Mission

- ❖ Disseminate the knowledge of a diverse group of students by providing solutions through contemporary curriculum.
- ❖ Creating a talent pool of faculty in diverse domains through continuous training.
- ❖ Domain and transferable skill development for holistic personality of students and employability.
- ❖ Inculcating values and Ethics for effective professional practice.

DEPARTMENT OF COMPUTER SCIENCE

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B-Tech Computer Science(aiml) will:

- PEO1.** Pursue higher education in computer science, Artificial Intelligence, Machine Learning Data Science, or allied areas of specialization.
- PEO2.** Employed as a productive and valued professional in industry/teaching/ research.
- PEO3.** Engaged in innovation and deployment as a successful entrepreneur.
- PEO4.** Adapt evolving technologies in the core or allied areas by participating in continuing education programs for lifelong learning.

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of B-Tech. Computer Science will be able to:

- PO1 Knowledge:** To study as well as apply concepts, theories, and practices across the disciplines to gain the foundational knowledge.
- PO2 Problem Analysis:** To identify, analyze and evaluate various experiences and perspectives using foundational disciplinary knowledge for substantiated conclusions.
- PO3 Design/Development of solutions:** To design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4 Modern tool usage:** To create, select, and apply appropriate techniques, resources and modern tools with an understanding of the limitations.
- PO5 Environment and Sustainability:** Understand the issues of environmental contexts and demonstrate the knowledge for sustainable development.
- PO6 Ethics and Society:** Apply the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities under moral dimensions.
- PO7 Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, to manage projects and finance in multidisciplinary settings.
- PO8 Effective Communication:** To develop proficiency and efficiency In communicating by connecting people, ideas, books, media, and technology.
- PO9 Life-long learning:** Recognize the need for and acquire the ability to engage in independent and life-long learning in the broadest context of socio-technological changes.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program, the graduates of B-Tech. Computer Science program students will be able to:

- PSO1.** Apply the knowledge of Maoundation, Business Management and Information Technology to the solutions of real world problems.
- PSO2.** Analyze, Design and Develop solutions in real time in the domains of technical, managerial, economical and social constraints by using current technologies in Information Management, Software Engineering, Platform Based Development, and Computer Networks skills.
- PSO3.** Use innovative ideas to create better environment in order to solve complex problems in the domains of Information Management, Software Engineering, Platform Based Development and Computer Networks for the excellence of an individual and society and apply appropriate techniques, resources, and modern tools to complex real time problems in the domains of Information Management, Software Engineering, Platform Based Development and Computer Networks.



MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati 517 102

SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

Certificate

This is to certify that the project report entitled “**Forest Wild fire and Smoke Detection using deep learning**” is the bona fide work carried out and submitted by

(KURUVA RAKSHITHA (23102A010433))

in the Department of **Computer Science, School of computing** of **Mohan Babu University**, Tirupati in partial fulfillment of the requirements for the award of the B-Tech of **Computer Science** during 2025-26.

Supervisor

Dr. K. Munisankar
Assistant Professor ,
Dept of Computer Science, MBU,
Tirupati.

Head Of Dept.

Dr. K. Kumara Swamy Naidu
Associate Professor & Head,
Dept of Computer Science
MBU, Tirupati.

Date:

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

This acknowledgement transcends the reality of formality when I would like to express my deep gratitude and respect to all those people behind the screen who guided, inspired and helped me for the completion of my project work.

I express my deep sense of gratitude to our beloved chancellor **Dr. M. Mohan Babu**, Padma Shri awardee for his encouragement throughout the program.

I express my deep sense of gratitude to our beloved Vice-Chancellor **Prof. P. V. Ramana**, for his encouragement throughout the Program.

We owe our gratitude and special thanks to the Dean **Dr. M. Balaji**, for his special encouragement throughout the program.

I am extremely thankful to, **Dr. K. Kumara Swamy Naidu**, HOD, and Department of **Computer Science**, for all provisions made and for her constant encouragement throughout my work.

I wish to express my deep sense of gratitude to my Project Supervisor **Dr. K. Muni Sankar**, Assistant Professor, Department of Computer Science for extending his valuable co-operation, moral support, kind attention, guidance, suggestions and encouragement to complete my Project Work successfully.

I thank all my beloved **Faculty**, Department of Computer Science for giving their valuable suggestions and maximum co-operation.

I owe a deep sense of gratitude to my beloved **Parents** in extending their moral support in this Endeavour. I would like to thank all my **friends** who extended their help, encouragement and moral support either directly or indirectly in completing my project work.

(KURUVA RAKSHITHA)

DECLARATION

I, **KURUVA RAKSHITHA** Hereby declare that, the project entitled “**Forest Wild fire and Smoke Detection using deep learning**” developed by me at **MOHANBABU UNIVERSITY, Tirupati** during the Academic year 2025- 2026 and submitted to the department of Computer Science, School of computing, MOHAN BABU UNIVERSITY for partial fulfilment for the award of department of Computer Science (B-Tech).

I also declare that, the project is resulted by my own effort and that it has not been copied from anyone and not been submitted by anybody in any of the University or Institution or Research Centre.

Place: Tirupathi

Date:

KURUVA RAKSHITHA

(23102A010433)

ABSTRACT

Forest wildfires rank among the most destructive natural hazards, causing severe damage to ecosystems and threatening human lives. Detecting smoke and fire at an early stage is essential to minimize losses and support rapid emergency response. This project explores the application of deep learning techniques to identify smoke and wildfire occurrences in forest environments. The primary goal is to design an automated detection system capable of accurately recognizing fire-related patterns, even in complex and changing surroundings.

To achieve this, advanced object detection models such as YOLOv8 and YOLOv9 (You Only Look Once) are employed. These models are widely recognized for their balance of speed and precision, making them suitable for real-time monitoring tasks. In particular, YOLOv9 enhances detection efficiency, allowing continuous observation of large forest regions where manual monitoring is impractical and prone to error. The proposed system analyzes visual data obtained from sources like surveillance cameras, unmanned aerial vehicles (drones), and satellite imagery to identify early indicators of fire and smoke. The models are trained using a diverse dataset that includes wildfire scenarios under different environmental conditions. With their improved feature extraction capabilities, YOLOv8 and YOLOv9 can effectively detect and classify fire-related patterns, even in situations involving poor lighting, weather variations, or partial obstructions from vegetation.

By automating the detection process, the system aims to significantly shorten the time between fire ignition and response initiation. This can help protect human life, wildlife, and forest resources. Furthermore, the solution can be integrated into real-time monitoring frameworks, enabling faster decision-making and efficient allocation of emergency resources in areas vulnerable to wildfires.

Keywords: Forest wildfires, early warning systems, smoke detection, fire detection, deep learning, YOLOv8, YOLOv9, object detection, real-time surveillance, drones, satellite imagery, automation, environmental variability, emergency response, resource management.

Table of contents

Title	Page No
Vision & Mission	ii
Certificate	vii
Acknowledgement	viii
Declaration	ix
Abstract	x

S.no	Title	Page no
1	Introduction	
	1.1 objective of project	1
	1.2 problem statement	1
	1.3 motivation	2
	1.4 scope	2
	1.5 project introduction	3
2	Literature Review	4
3	System Analysis	
	3.1 Existing system	10
	3.2 Advantages of the Proposed system	11
	3.3 Proposed system	11
	3.4 Disadvantages of the Existing system	11
	3.5 Project Flow	12
4	System Design	
	4.1 Introduction of Input Design	14
	4.2 Output Design	15
	4.3 UML Diagrams	16
	4.4 DFD Diagrams	24
5	Technologies and Tools	
	5.1 Programming Languages	28
	5.2 Software Requirements	28
	5.3 Hardware Requirements	29
6	Implementation	
	6.1 Data Collection	30
	6.2 Model Training	31
	6.3 Algorithms Used	32
	6.4 Code Snippets	33

7	System Study And Testing	
	7.1 Feasibility Study	36
	7.2 Types of Tests	37
8	Results and Discussion	
	8.1 Output Screens	41
	8.2 Result Analysis	45
9	Conclusion and Future work	
	9.1 Conclusion	52
	9.2 Achievements of the Project	52
	9.3 Limitations	52
	9.4 Future Enhancements	53
10	References	54

Appendix-A: Plagiarism Report

Appendix-B: Time and Cost Analysis

your project work

Appendix-C: Conference paper

Appendix-D: Sample Source Code

CO-PO-PSO Mapping table

Short Bio-data of the Student

List of Figures

S.no	Title	Page no
1.	Fig 3.5.1 Project Flow	12
2.	Fig 4.3.1 Use Case Diagram	16
3.	Fig 4.3.2 Class Diagram	17
4.	Fig 4.3.3 Sequence Diagram	18
5.	Fig 4.3.4 Collaboration Diagram	19
6.	Fig 4.3.5 Deployment Diagram	20
7.	Fig 4.3.6 Component Diagram	21
8.	Fig 4.3.7 Activity Diagram	22
9.	Fig 4.3.8 ER Diagram	23
10.	Fig 4.4.1 Context Diagram	24
11.	Fig 4.4.2 DFD Level-1 Diagram	25
12.	Fig 4.4.3 DFD Level-2 Diagram	26
13.	Fig 8.2.1 home page	41
14.	Fig 8.2.2 About page	42
15.	Fig 8.2.3 login page	42
16.	Fig 8.2.4 Registration page	43
17.	Fig 8.2.5 Upload page	43
18.	Fig 8.2.6 prediction page	44
19.	Fig 8.2.7 output page	44

Chapter – 1

INTRODUCTION

1.1 Objective of Project:

The main aim of this project is to design and implement an automated system capable of detecting forest fires and smoke at an early stage using deep learning methods, with a focus on the YOLOv9 algorithm. The system is intended to perform real-time surveillance over extensive forest regions by processing visual data collected from multiple platforms, including surveillance cameras, drones, and satellite sources.

By utilizing the fast and accurate object detection capabilities of YOLOv9, the system seeks to identify signs of smoke and fire promptly and reliably. This early detection mechanism is expected to minimize delays in emergency response, thereby reducing potential damage. Ultimately, the project contributes to environmental protection, improved public safety, and efficient management of wildfire incidents in high-risk areas.

1.2 Problem Statement:

Forest wildfires and smoke pose significant environmental and safety hazards, with delayed detection often leading to devastating consequences. Traditional methods of monitoring, such as manual surveillance, are inefficient and slow, making it difficult to respond in time. There is a need for an automated, real-time detection system that can quickly identify the presence of smoke and wildfire in vast forested areas. This project aims to address this issue by leveraging deep learning, specifically the YOLOv9 algorithm, for accurate and rapid detection of wildfires and smoke, enabling faster response times and minimizing the destruction caused by such natural disasters.

1.3 Motivation:

Wildfires pose significant threats to ecosystems, human safety, and infrastructure, often leading to irreversible environmental damage. Traditional methods of wildfire detection, such as manual monitoring and visual surveillance, are time-consuming and prone to errors. Timely detection is crucial to mitigate the effects of wildfires and ensure prompt response actions. The motivation behind this project is to leverage deep learning algorithms, particularly YOLOv9, for real-time detection of smoke and fire in vast forested areas. By automating the detection process, we aim to enhance the speed and accuracy of identifying wildfires, ultimately saving lives, preserving ecosystems, and minimizing damage to natural resources.

1.4 SCOPE:

The scope of this project encompasses the development and implementation of an automated wildfire and smoke detection system using deep learning algorithms, specifically YOLOv8, YOLOv9. The system will focus on the following key areas:

- **Data Collection and Preprocessing:** Collecting and preparing a diverse dataset of images and video feeds depicting wildfires and smoke in various environmental conditions.
- **Model Development:** Training the YOLOv8, YOLOv9 model to accurately detect smoke and wildfire from images and videos, ensuring high precision and speed for real-time detection.
- **Integration:** Deploying the system to process data from multiple sources such as surveillance cameras, drones, and satellite imagery for widespread coverage.
- **Scalability and Adaptability:** Ensuring the system can be adapted to different terrains and integrated into existing environmental monitoring frameworks.
- The system will be designed for forested areas, but its application can extend to other fire-prone regions, enhancing global wildfire detection efforts.

1.5 PROJECT INTRODUCTION:

Wildfires are one of the most devastating natural disasters, affecting millions of acres of land every year, destroying ecosystems, wildlife, and human settlements. Their rapid spread and unpredictability make it difficult to manage and control, resulting in catastrophic consequences for both the environment and human safety. Early detection of smoke and fire is vital in minimizing the damage caused by wildfires. However, traditional wildfire detection systems, such as human patrols, satellite imagery, and manual monitoring, are often slow, costly, and inefficient. These systems struggle to detect wildfires in real-time, especially in dense forests, mountainous areas, and regions with difficult terrain.

This project addresses the challenges of early wildfire detection by leveraging advanced deep learning techniques, specifically the YOLOv9 (You Only Look Once) algorithm. YOLOv9 is a cutting-edge object detection model known for its ability to detect objects in real-time with high accuracy and speed. It is particularly well-suited for applications where quick and efficient detection is crucial, such as monitoring large forested areas for signs of wildfire or smoke. The YOLOv9 model processes images and video feeds from surveillance cameras, drones, and satellites, making it an ideal solution for continuous, real-time monitoring.

The system developed in this project is designed to automatically identify the presence of smoke and fire

in forested areas. The model is trained on a diverse dataset of wildfire and smoke images captured under various environmental conditions, including different lighting, weather, and vegetation scenarios.

YOLOv9's robust capabilities allow it to detect and classify smoke and fire even when the conditions are challenging, such as when visibility is obstructed by trees or fog.

By automating the detection process, the system significantly reduces the time required to detect wildfires, allowing for faster response times. This proactive approach to wildfire detection could help mitigate damage to property, wildlife, and forest resources, while also enabling quicker evacuation of affected areas, saving lives in the process. Furthermore, the system can be integrated into real-time monitoring platforms used by forestry agencies, emergency responders, and local authorities, providing them with accurate and timely information to aid in decision-making and resource allocation.

In conclusion, this project aims to create an efficient, automated wildfire detection system that can be deployed in various real-world environments to provide early warnings and mitigate the disastrous effects of wildfires.

Summary

This project focuses on developing an automated forest wildfire and smoke detection system using advanced deep learning techniques, particularly the YOLOv9 algorithm. The main objective is to enable real-time monitoring and early detection of fire and smoke across large forest areas using data from surveillance cameras, drones, and satellites.

The problem addressed is the delay and inefficiency of traditional wildfire detection methods, such as manual monitoring and basic surveillance systems, which often result in severe environmental and economic damage. To overcome this, the proposed system uses the high-speed and accuracy capabilities of YOLOv9 to detect wildfire indicators quickly and reliably.

The motivation behind the project stems from the urgent need to protect ecosystems, human lives, and infrastructure from the devastating effects of wildfires. By automating the detection process, the system reduces human effort, minimizes errors, and ensures faster emergency response.

The scope of the project includes data collection and preprocessing, model development using YOLOv8 and YOLOv9, and integration with real-time monitoring systems. It also emphasizes scalability, allowing the system to adapt to different environments and extend beyond forest areas to other fire-prone regions.

Chapter – 2

LITERATURE SURVEY

2.1 Related work:

Forest wildfire and smoke detection systems have evolved significantly over time, moving from traditional manual methods to advanced AI-based automated systems. Existing systems can be broadly classified into the following categories.

2.2 Review of Related Research Papers:

1. Author: Ying Zheng et al.

Title: A Forest Fire Smoke Detection Model Combining CNN and Vision Transformer

Published Year: 2023

Link: <https://www.frontiersin.org/articles/10.3389/ffgc.2023.1136969/full>

Pros

- Hybrid CNN + Transformer model (SR-Net)
- High accuracy (~96.9%)
- Works well with satellite imagery

Cons

- Requires large dataset
- Model complexity

Outcomes

- Outperformed ResNet, AlexNet, MobileNet
- Effective for early smoke detection

2. Author: Chayma Bahhar et al.

Title: Wildfire and Smoke Detection Using Staged YOLO Model and Ensemble CNN

Published Year: 2023

Link: <https://www.mdpi.com/2079-9292/12/1/228>

Pros

- Combines YOLO + CNN ensemble
- High accuracy (~99%)
- Good localization performance

Cons

- Needs high-quality training data
- Multi-stage increases complexity

Outcomes

- Strong detection accuracy and real-time performance
- Effective pipeline for classification + detection

3. Author: V. E. Sathishkumar et al.

Title: Forest Fire and Smoke Detection Using Deep Learning-Based Learning Without Forgetting

Published Year: 2023

Link: <https://fireecology.springeropen.com/articles/10.1186/s42408-022-00165-0>

Pros

- Uses continual learning
- Adapts to new datasets

Cons

- Complex training process
- Requires careful tuning

Outcomes

- Maintains performance over time
- Reduces retraining cost

4. Author: Luhao He et al.

Title: Research and Application of Deep Learning Object Detection Methods for Forest Fire Smoke Recognition

Published Year: 2025

Link: <https://www.nature.com/articles/s41598-025-98086-w>

Pros

- Latest deep learning models
- Handles complex environments

Cons

- High computational demand

Outcomes

- Improved detection accuracy in real-world conditions

5. **Author:** Multiple (MDPI Study)

Title: YOLO-Based Models for Smoke and Wildfire Detection in Ground and Aerial Images

Published Year: 2024

Link: <https://www.mdpi.com/2571-6255/7/4/140>

Pros

- Uses YOLOv7x
- Works on drone + ground images

Cons

- Accuracy varies with environment

Outcomes

- YOLOv7x achieved ~80% mAP
- Faster detection speeds

6. **Author:** (ScienceDirect Research Team)

Title: A Dual Track YOLO-Based Network with Multi-Scale Feature Fusion for Fire and Smoke Detection

Published Year: 2026

Link: <https://www.sciencedirect.com/science/article/pii/S1051200425008917>

Pros

- Combines CNN + Swin Transformer
- Multi-scale feature extraction

Cons

- Computationally expensive
- Complex architecture

Outcomes

- Improved feature extraction and detection accuracy
- Better performance than traditional models

7. **Author:** Uma Meleti, Abolfazl Razi

Title: Obscured Wildfire Flame Detection by Temporal Analysis of Smoke Patterns

Published Year: 2023

Link: <https://arxiv.org/abs/2307.00104>

Pros

- Detects hidden fires (under smoke/trees)
- Uses temporal video analysis

Cons

- Requires video data
- Computational cost

Outcomes

- High precision (~92%)
- Effective for drone-based monitoring

8. **Author:** Chong Wang et al.

Title: Wildfire Smoke Detection with Cross Contrast Patch Embedding

Published Year: 2023

Link: <https://arxiv.org/abs/2311.10116>

Pros

- Transformer-based approach
- Handles subtle smoke features

Cons

- Complex training
- Requires large dataset

Outcomes

- Improved detection of faint smoke
- Better feature representation

9. **Author:** Siddhant Baldota et al.

Title: Multimodal Wildland Fire Smoke Detection (SmokeyNet)

Published Year: 2022

Link: <https://arxiv.org/abs/2212.14143>

Pros

- Uses multimodal data (image + weather)
- Faster detection

Cons

- Requires multiple sensors
- Data integration complexity

Outcomes

- Early detection within minutes
- Improved real-time monitoring

10. **Author:** Ziliang Hong et al.

Title: Wildfire Detection via Transfer Learning: A Survey

Published Year: 2023

Link: <https://arxiv.org/abs/2306.12276>

Pros

- Comprehensive survey
- Covers multiple DL models

Cons

- Not an implementation paper

Outcomes

- Identifies best models (Swin Transformer, ConvNext)
- Highlights future research directions

RESEARCH GAP IDENTIFICATION:

The analysis of existing forest wildfire and smoke detection systems reveals several important research gaps that need to be addressed. Most current approaches struggle with accurately detecting early-stage fires, especially when smoke is thin or partially obscured by environmental factors such as fog, dust, or dense vegetation. Many models are trained on limited and non-diverse datasets, which reduces their ability to generalize across different forest types and weather conditions. There is also a lack of robust real-time detection systems that can operate efficiently in remote areas with limited connectivity and power resources. Additionally, existing methods often produce high false positives and false negatives, affecting reliability and response time. Integration of multi-modal data sources, such as combining visual data with sensor or satellite inputs, is still underexplored. Furthermore, scalability and cost-effectiveness remain

major challenges for large-scale deployment. These gaps highlight the need for more adaptive, accurate, and efficient wildfire detection systems that can perform consistently in diverse and challenging environments.

S.No	Existing Paper Name	Limitations / Disadvantages	Overcomes from New Project (Orderista)
1.	CNN-based Fire Detection System	High false positives due to fog, clouds	Uses improved model with better feature extraction to reduce false alarms
2.	Satellite-based Wildfire Detection	Delay in detection due to low revisit time	Real-time detection using live camera/video input
3.	Sensor-based Fire Detection	Limited coverage and expensive deployment	Scalable system using cameras and AI models
4.	Traditional Image Processing Methods	Low accuracy in complex environments	Deep learning model improves detection accuracy
5.	Smoke Detection using Basic Algorithms	Difficulty in detecting thin smoke	Enhanced training dataset improves smoke detection
6.	Drone-based Monitoring Systems	High cost and maintenance issues	Cost-effective solution without heavy dependency on drones
7.	Human Monitoring Systems	Prone to human error and slow response	Automated alerts ensure faster and reliable detection

Chapter – 3

SYSTEM ANALYSIS

3.1 Existing System

Existing systems for wildfire and smoke detection primarily rely on traditional methods such as satellite imaging, manual monitoring, and fixed sensors to detect fires. These systems often suffer from delayed detection, high costs, and limited coverage, especially in vast forested areas. Some advanced systems use infrared sensors and remote sensing technologies, but they are usually expensive and not always accurate in distinguishing between fire, smoke, and other environmental factors. Additionally, these systems lack real-time capabilities and automation, leading to slower response times. Overall, existing systems are often inefficient, making timely fire detection and intervention challenging.

3.2 Disadvantages of the Existing System

Disadvantages of existing wildfire and smoke detection systems include:

1. **Delayed Detection:** Traditional methods such as satellite imaging and manual monitoring often result in slow detection, delaying response times and increasing the damage caused by wildfires.
2. **High Costs:** Systems like infrared sensors and remote sensing technologies are expensive to install and maintain, making them less accessible for widespread use, especially in remote or large areas.
3. **Limited Coverage:** Fixed sensors and manual monitoring systems have limited coverage, leaving vast forested areas unmonitored or inadequately covered, increasing the risk of undetected fires.
4. **Lack of Real-Time Capabilities:** Most existing systems do not provide real-time detection, which is crucial for immediate intervention and timely resource deployment in wildfire-prone areas.
5. **Inaccurate Detection:** Some systems struggle to differentiate between actual fire/smoke and other environmental factors, leading to false alarms or missed detections.
6. **Limited Automation:** Many existing systems lack automation, requiring human intervention for analysis and decision-making, which slows down the process of responding to potential wildfires.

3.3 Proposed System

The proposed system leverages deep learning, specifically the YOLOv8, YOLOv9 algorithm, for real-time detection of smoke and wildfires in forested areas. Unlike traditional methods, it processes images and video feeds from drones, surveillance cameras, and satellites to identify early signs of fire and smoke with high accuracy. The system automates monitoring, reducing reliance on manual observation, and generates instant alerts to facilitate rapid response. By handling diverse environmental conditions and large-scale forest areas efficiently, the system aims to minimize environmental damage, protect human and wildlife lives, and provide a scalable, cost-effective solution for continuous wildfire surveillance.

3.4 Advantages of the Proposed System

Advantages of the proposed system include:

1. **Quick Detection:** The system uses the YOLOv8, YOLOv9 algorithm for fast detection, allowing for immediate identification of smoke and fire, enabling quick response and intervention to mitigate wildfire spread.
2. **High Accuracy:** YOLOv8, YOLOv9's advanced deep learning capabilities offer high accuracy in detecting and classifying smoke and fire, even under challenging conditions such as varying weather, lighting, or partial obstructions by trees and vegetation.
3. **Automation:** By automating the monitoring process, the system reduces the need for manual observation, which is resource-intensive and prone to human error, enhancing operational efficiency and reducing the time between detection and response.
4. **Scalability:** The system can be deployed in large-scale forested areas and integrated with drones, surveillance cameras, and satellites, allowing for widespread coverage and scalability to cover vast regions at a fraction of the cost of traditional methods.

Cost-Effective: Compared to infrared sensors and remote sensing technologies, the proposed system offers a more cost-effective solution for wildfire detection by utilizing existing video feeds from drones, cameras, and satellites, without the need for expensive infrastructure.

3.5 PROJECT FLOW

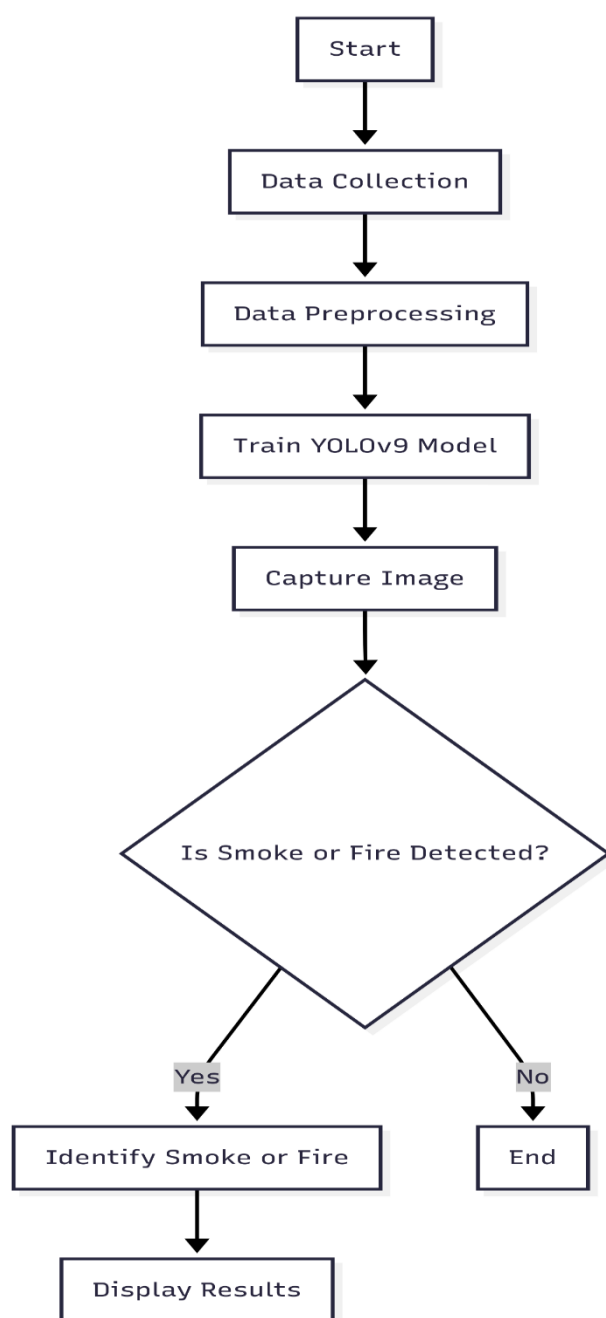


Fig 3.5.1 PROJECT FLOW

Summary

The existing wildfire and smoke detection systems rely on traditional methods such as satellite imaging, manual monitoring, and fixed sensors. These systems face several challenges, including delayed detection, high implementation costs, limited coverage, lack of real-time monitoring, and inaccurate detection, which can lead to severe damage due to late response.

To address these issues, the proposed system uses advanced deep learning techniques, specifically YOLOv8 and YOLOv9 algorithms, for real-time and automated detection of smoke and wildfire. It processes data from drones, cameras, and satellites to ensure faster and more accurate identification.

The proposed system offers several advantages, including quick detection, high accuracy, automation, scalability, and cost-effectiveness. By reducing human intervention and enabling continuous monitoring, it significantly improves response time and helps minimize environmental and human losses.

Chapter – 4

SYSTEM DESIGN

4.1 Introduction of Input Design:

In an information system, input refers to the raw data that is collected and processed to generate meaningful output. While designing input, developers must take into account the various input devices used, such as personal computers, MICR (Magnetic Ink Character Recognition), and OMR (Optical Mark Recognition).

The effectiveness of system output largely depends on the quality of the input provided. Therefore, properly designed input forms and screens should possess the following characteristics:

- They must efficiently fulfill their intended purpose, such as data storage, recording, and retrieval.
- They should ensure data accuracy and completeness.
- They must be simple, user-friendly, and easy to understand.
- They should emphasize consistency, clarity, and ease of use to guide user interaction.

These goals can be achieved by applying fundamental design principles, including:

- Identifying the exact input requirements of the system.
- Understanding how users interact with and respond to different form and screen elements.

Objectives for Input Design:

The main goals of input design include:

- Creating effective data entry methods and input procedures.
- Minimizing the amount of data entered wherever possible.
- Designing source documents or alternative methods for efficient data collection.
- Developing structured input formats such as data records, entry screens, and user interfaces.
- Implementing validation techniques and input controls to ensure data reliability.

4.2 Output Design:

Output design is a critical component of system development. It involves determining the type of information to be produced, establishing appropriate control mechanisms, and designing sample layouts for reports and displays.

Objectives of Output Design:

The key objectives of output design are:

- To create outputs that effectively serve their intended purpose while avoiding unnecessary information.
- To ensure that outputs meet the needs and expectations of end users.
- To provide the correct amount of information without overload.
- To present output in a clear format and deliver it to the appropriate users.
- To ensure timely availability of output to support informed decision-making.

4.3 UML Diagrams:

4.3.1 USE CASE DIAGRAM

A use case diagram is a behavioral diagram in the Unified Modeling Language (UML) that is developed during use-case analysis. It provides a visual representation of the system's functionality by illustrating the interactions between external users (actors) and the system.

The diagram highlights the different tasks or services (use cases) that the system offers, along with the relationships and dependencies among them. Its primary purpose is to identify which functions are carried out by the system and who (which actor) is involved in each function. It also clearly defines the roles and responsibilities of various actors interacting with the system.

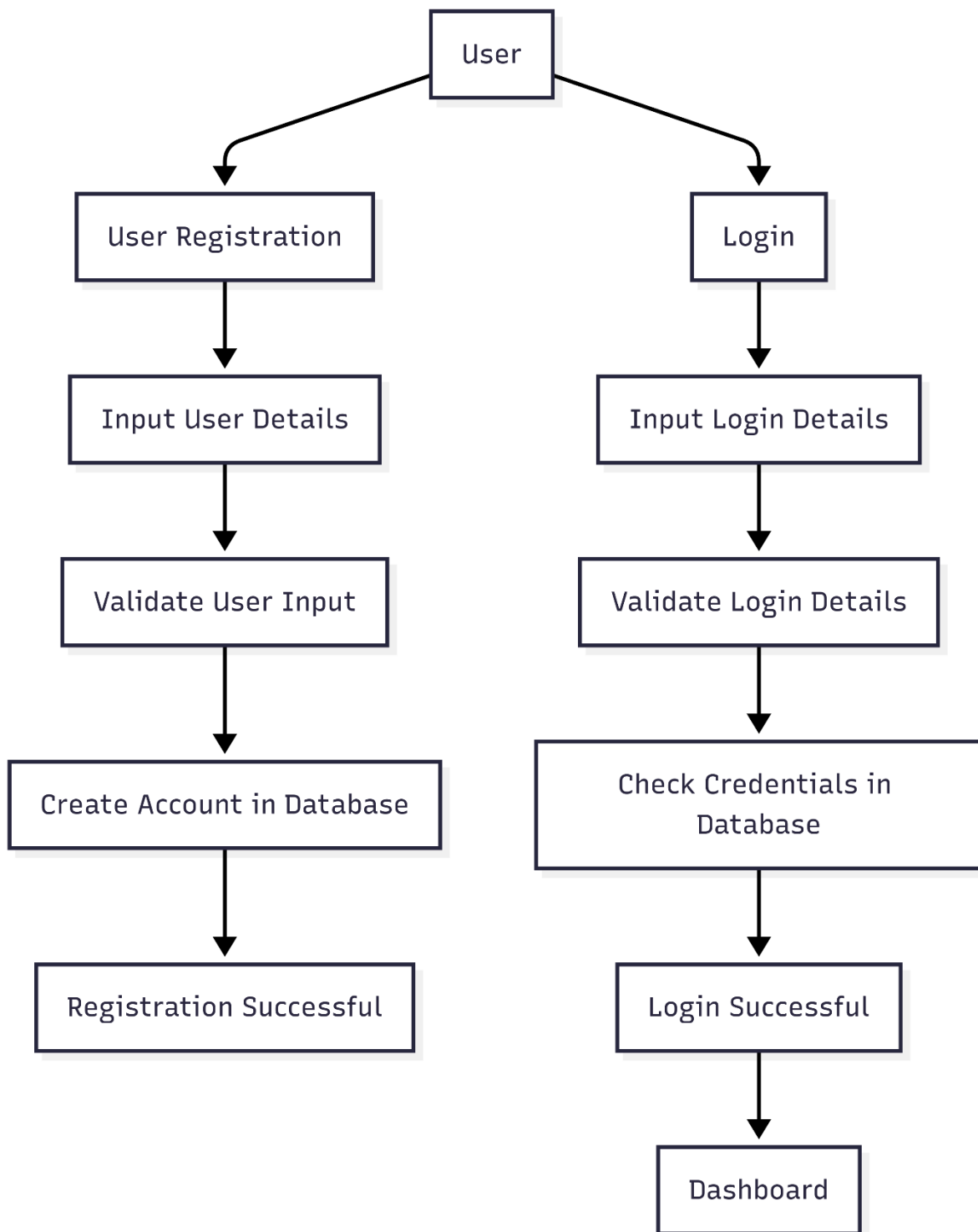


Fig 4.3.1 USE CASE DIAGRAM

4.3.2 CLASS DIAGRAM:

In software engineering, a class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that represents the overall structure of a system. It illustrates the system's classes along with their attributes, methods (operations), and the relationships between them. This diagram helps in understanding how data and functionality are organized within different classes and shows which class is responsible for storing specific information.

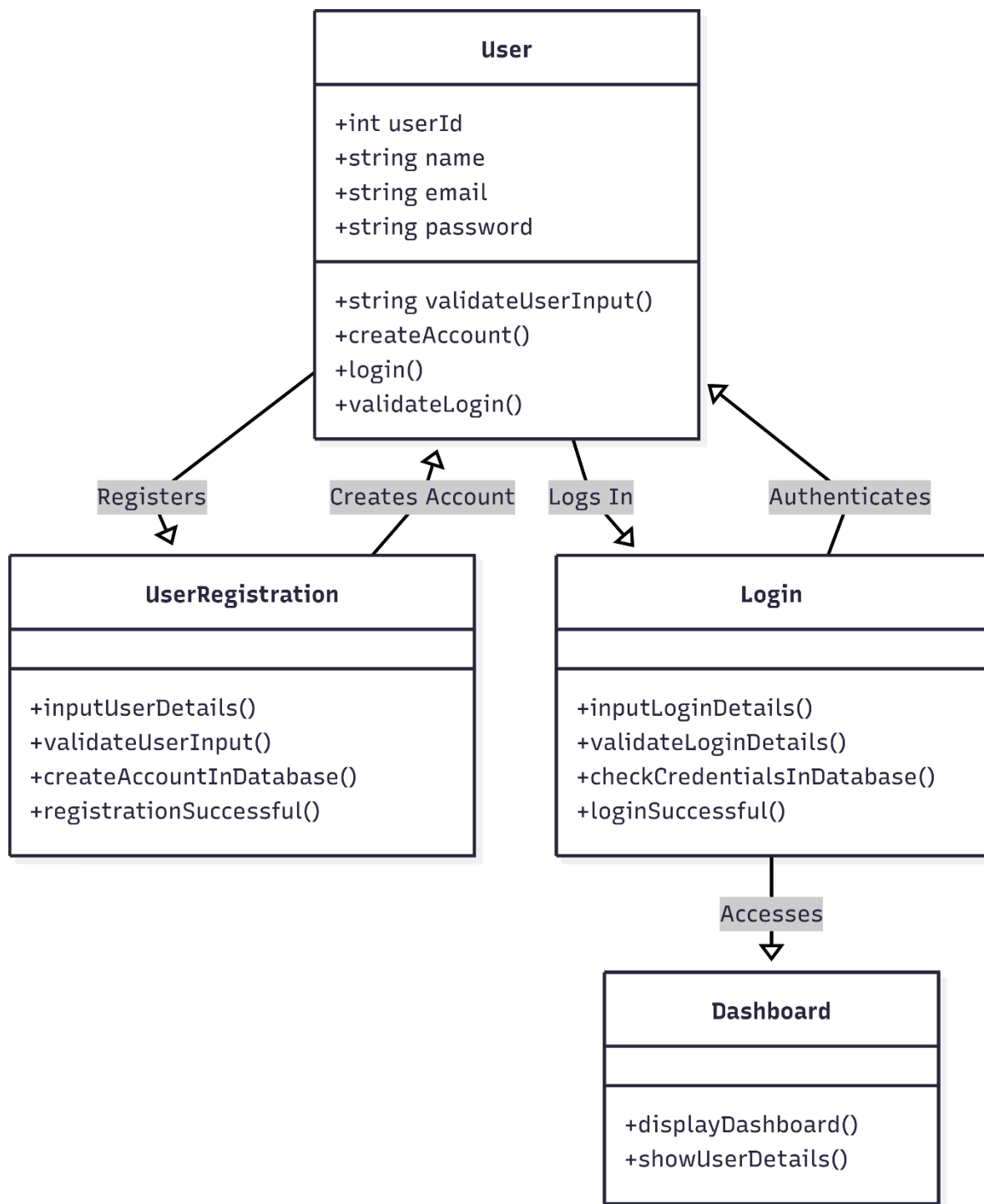


Fig 4.3.2 class diagram

4.3.3 SEQUENCE DIAGRAM

In the Unified Modeling Language (UML), a sequence diagram is a type of interaction diagram that illustrates how different components of a system communicate with each other over time. It represents the sequence in which messages are exchanged between processes or objects to perform a specific task. This diagram focuses on the chronological order of interactions, making it easier to understand the flow of operations within the system. Sequence diagrams are also referred to as event diagrams, scenario diagrams, or timing diagrams.

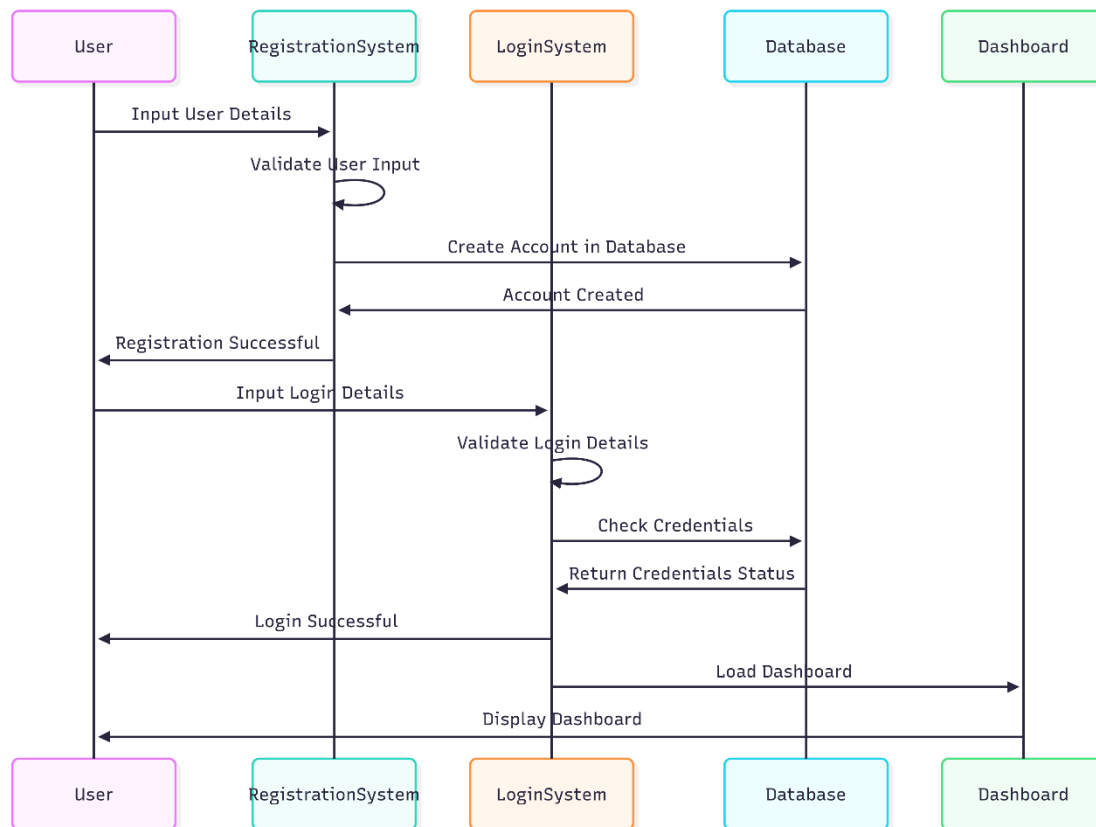


Fig 4.3.3 SEQUENCE DIAGRAM

4.3.4 COLLABORATION DIAGRAM:

A collaboration diagram, also known as a communication diagram in UML, illustrates how objects interact within a system. The sequence of method calls is represented using a numbering scheme, which indicates the order in which messages are passed from one object to another.

This diagram presents interactions similar to those shown in a sequence diagram; however, it differs in its focus. While sequence diagrams emphasize the time-based order of messages, collaboration diagrams highlight the structural arrangement and relationships between objects. In other words, they provide a clearer view of how objects are organized and connected while performing system operations.

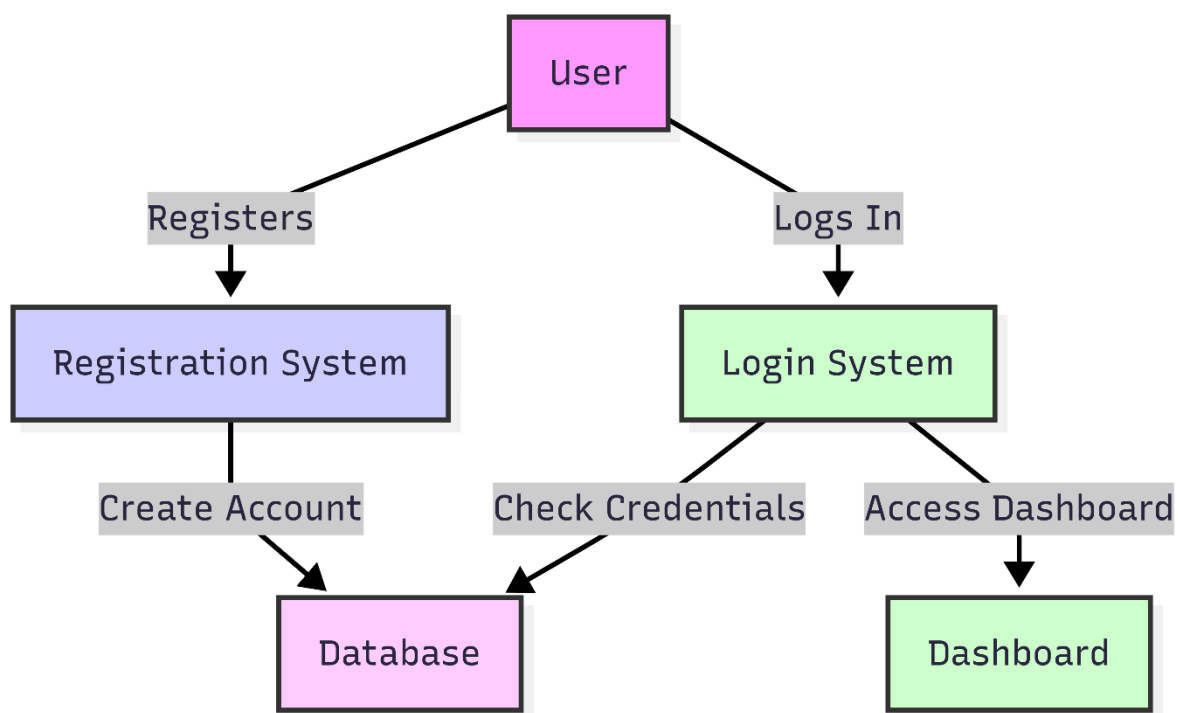


Fig 4.3.4 COLLABORATION DIAGRAM

4.3.5 DEPLOYMENT DIAGRAM

A deployment diagram illustrates the physical arrangement of a system, showing how software components are installed and executed on hardware. It is closely associated with component diagrams, as it demonstrates how those components are distributed across different devices.

The diagram is made up of nodes, which represent physical hardware elements such as servers, computers, or other devices where the application is deployed. It helps in understanding the system's infrastructure and how various parts of the application are hosted and connected.

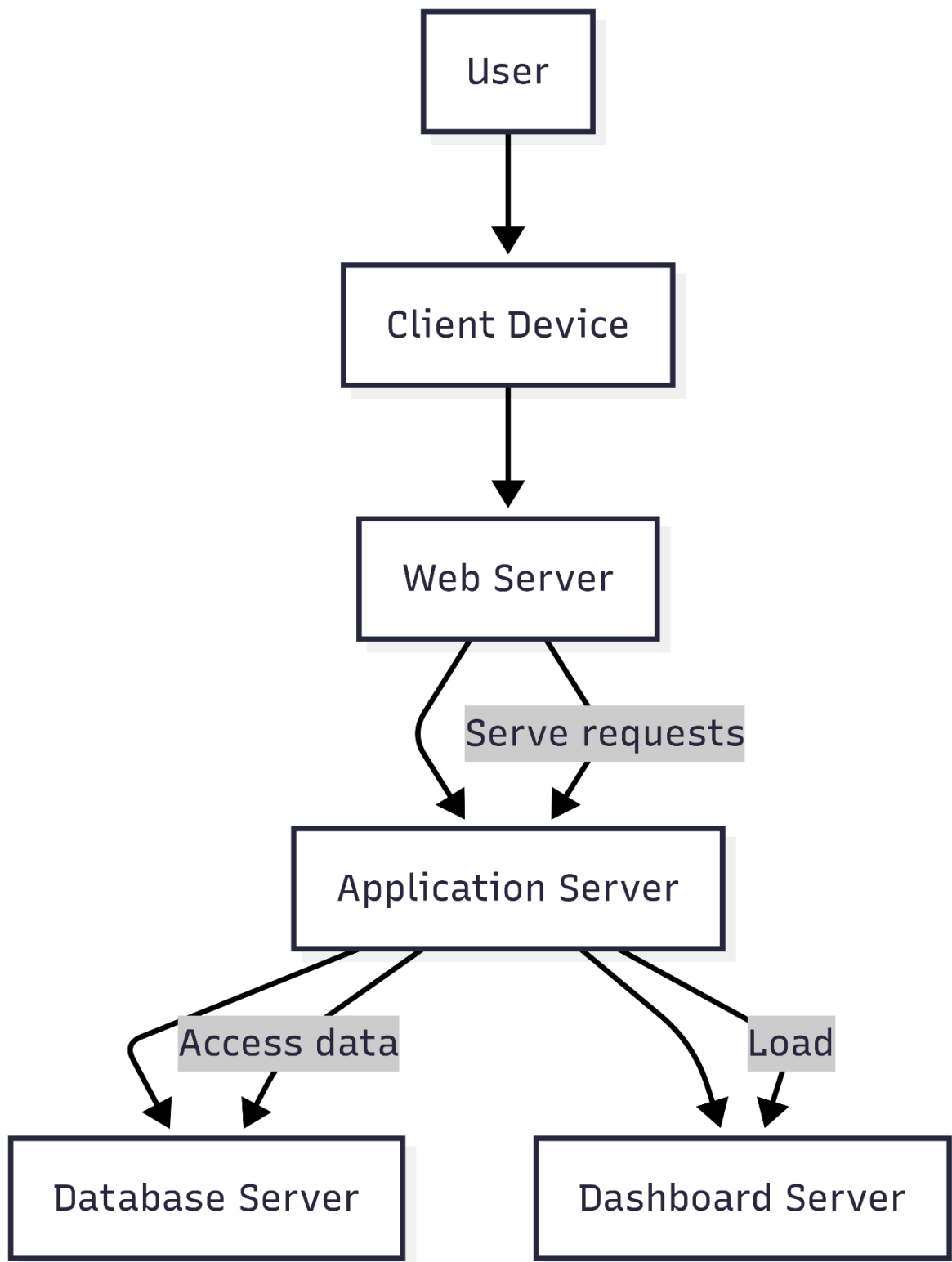


Fig 4.3.5 DEPLOYMENT DIAGRAM

4.2.6 COMPONENT DIAGRAM:

A component diagram, commonly referred to as a UML component diagram, illustrates how the physical components of a system are organized and interconnected. It provides a view of the system's implementation by showing how different modules or components are linked together.

These diagrams are useful for representing the structure of the system at a higher level and ensuring that all required functionalities are properly supported by the available components. They also help in verifying that the system design is complete and well-organized.

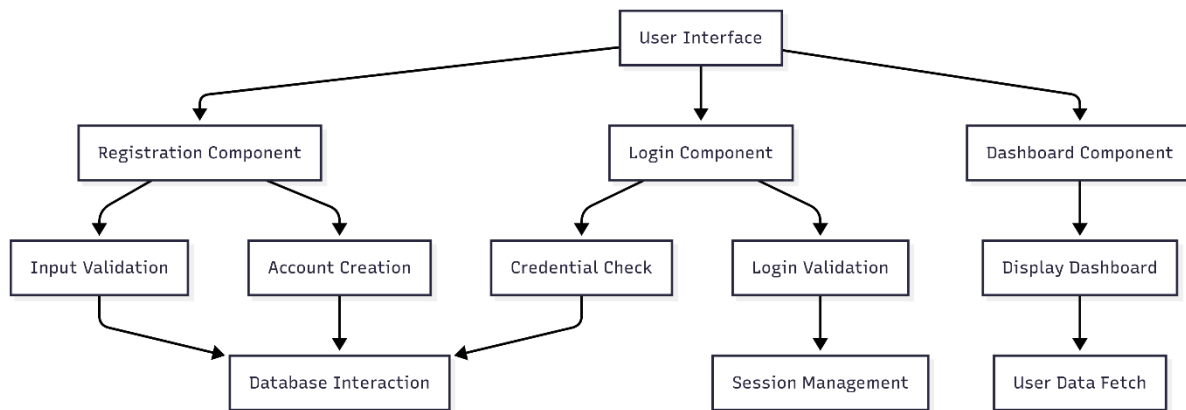


Fig 4.3.6 COMPONENT DIAGRAM

4.3.7 ACTIVITY DIAGRAM:

An activity diagram is a visual representation used to model the flow of activities and actions within a system. It illustrates step-by-step processes and supports concepts such as decision-making, looping, and parallel execution.

In the Unified Modeling Language (UML), activity diagrams are used to describe both business processes and system operations in a structured manner. They provide a clear view of how tasks are carried out and how control flows from one activity to another within the system.

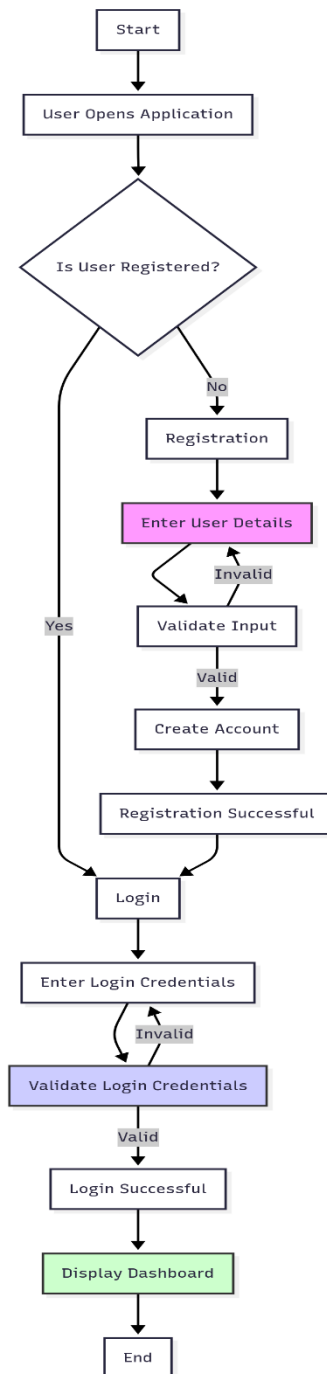


Fig 4.3.7 ACTIVITY DIAGRAM

4.3.8 ER DIAGRAM

An Entity–Relationship (ER) model is used to represent the structure of a database through a visual diagram known as an ER diagram. It provides a clear depiction of how data is organized within the system. An ER diagram illustrates the relationships between different entity sets, where an entity set refers to a collection of similar objects or items. Each entity is associated with attributes that define its properties. From a DBMS perspective, an entity can be represented as a table, while attributes correspond to the fields within that table. By showing the connections between entities and their attributes, an ER diagram presents a complete logical view of the database structure.

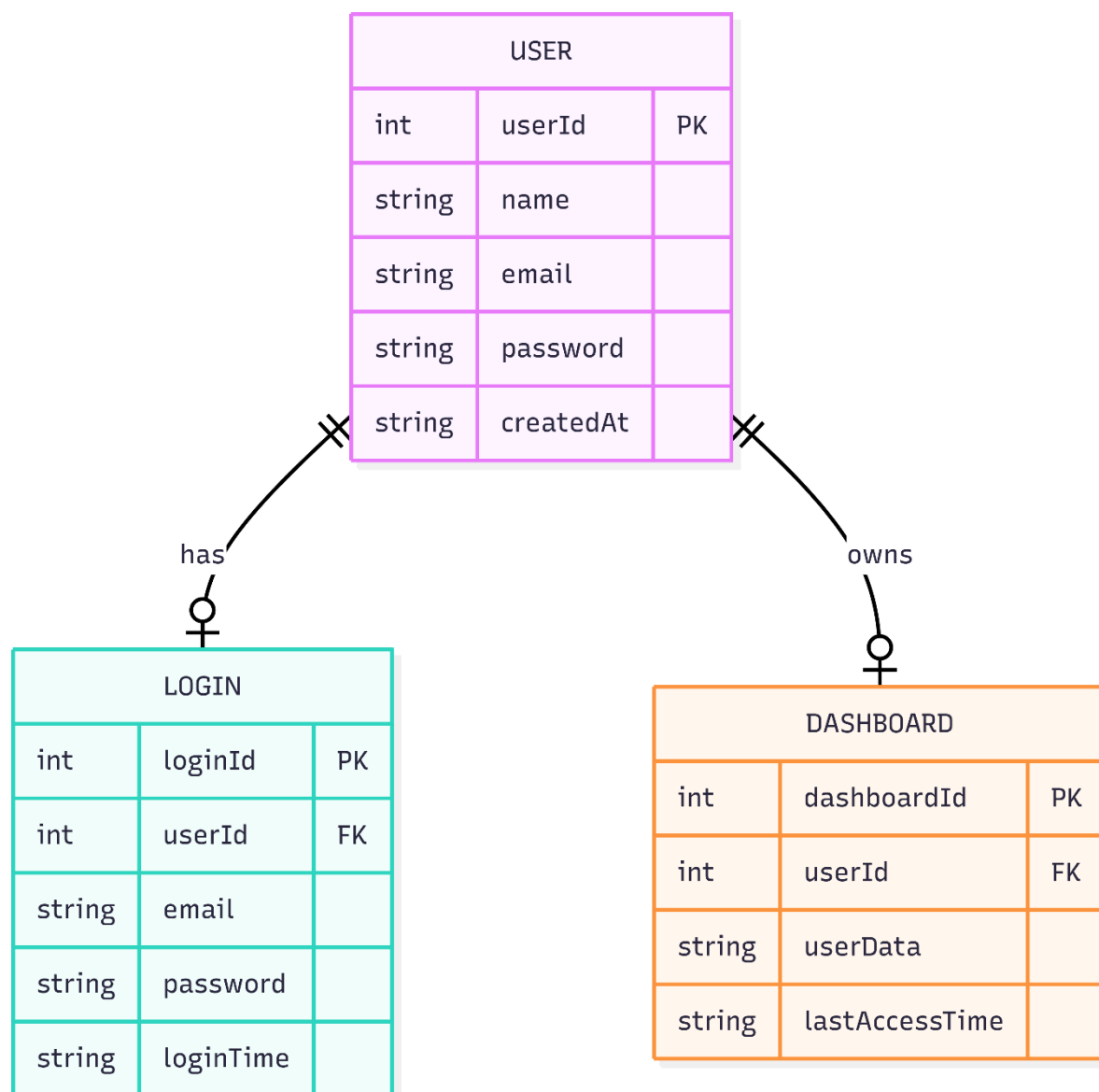


Fig 4.3.8 ER DIAGRAM

4.4 DFD DIAGRAM

A Data Flow Diagram (DFD) is a commonly used method for representing how data moves within a system. It provides a clear visual overview of system requirements by illustrating the flow of information. A DFD shows how data enters and exits the system, the processes that transform the data, and the locations where data is stored. The system represented can be manual, automated, or a combination of both.

The main purpose of a DFD is to define the overall scope and boundaries of a system. It also serves as an effective communication tool between system analysts and stakeholders, and can act as a foundation for analyzing and improving an existing system.

4.4.1 Context Diagram:

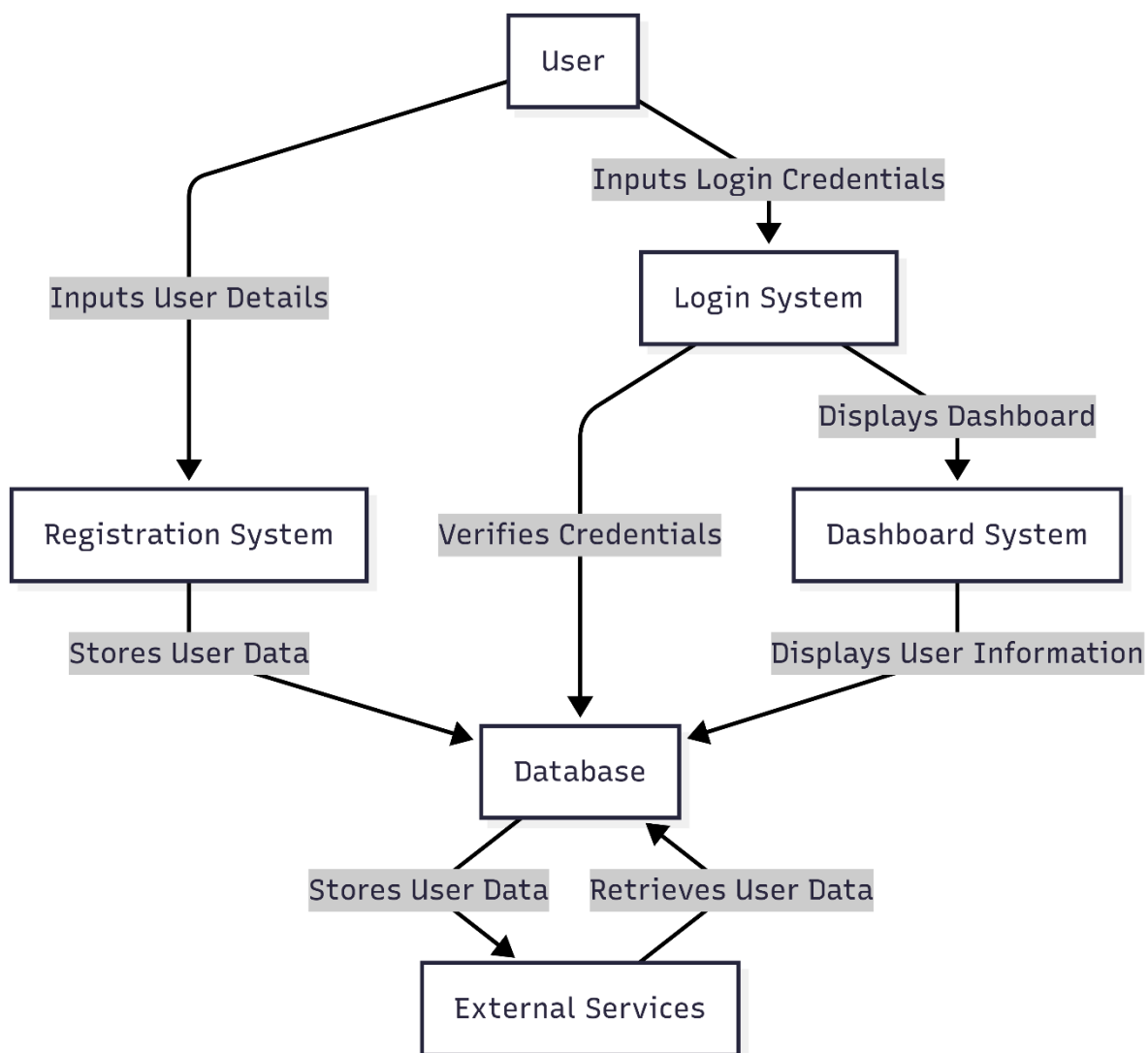


Fig 4.4.1 Context Diagram

4.2 DFD Level-1 Diagram:

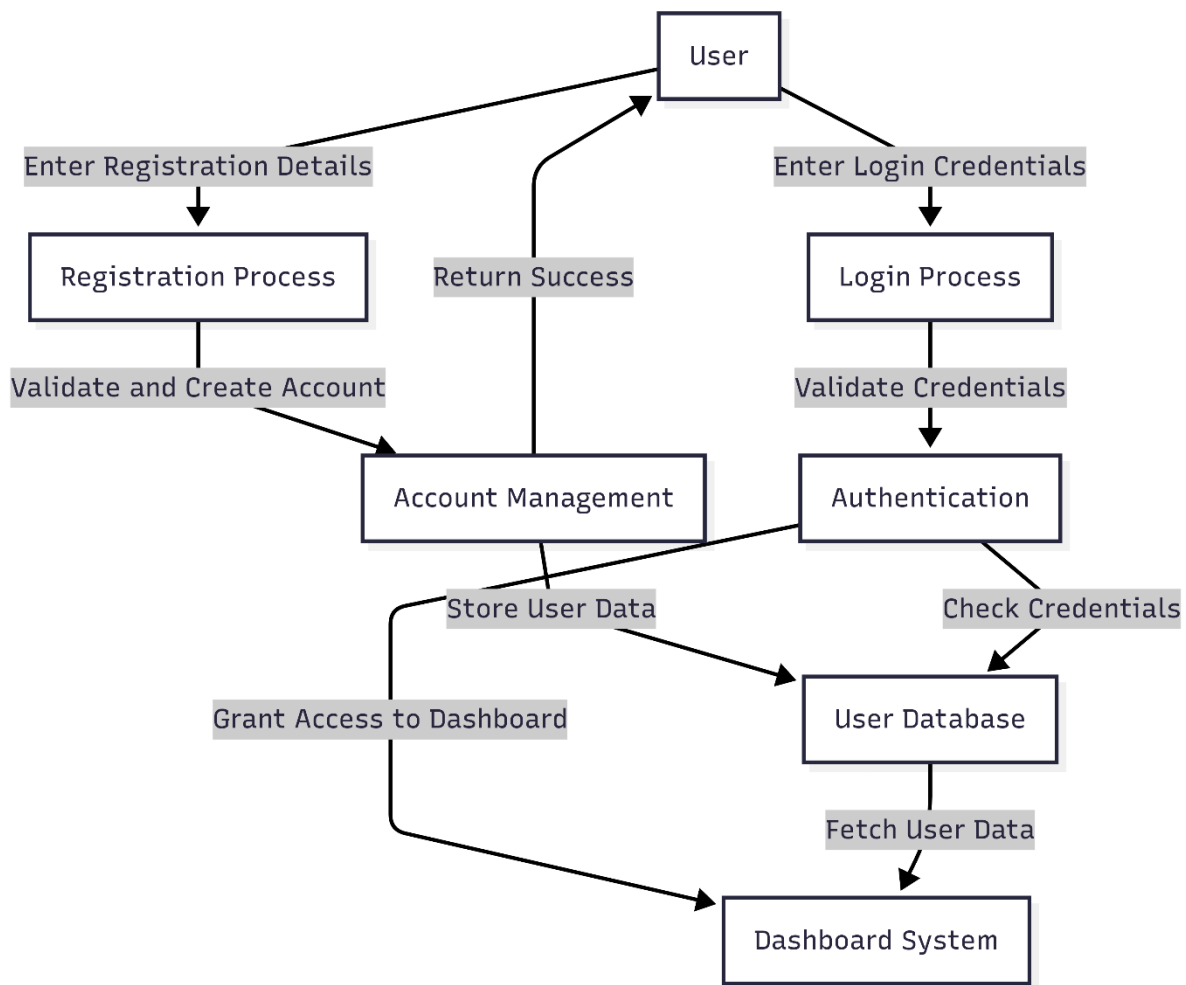


Fig 4.4.2 DFD Level-1 Diagram

4.3 DFD Level-2 Diagram:

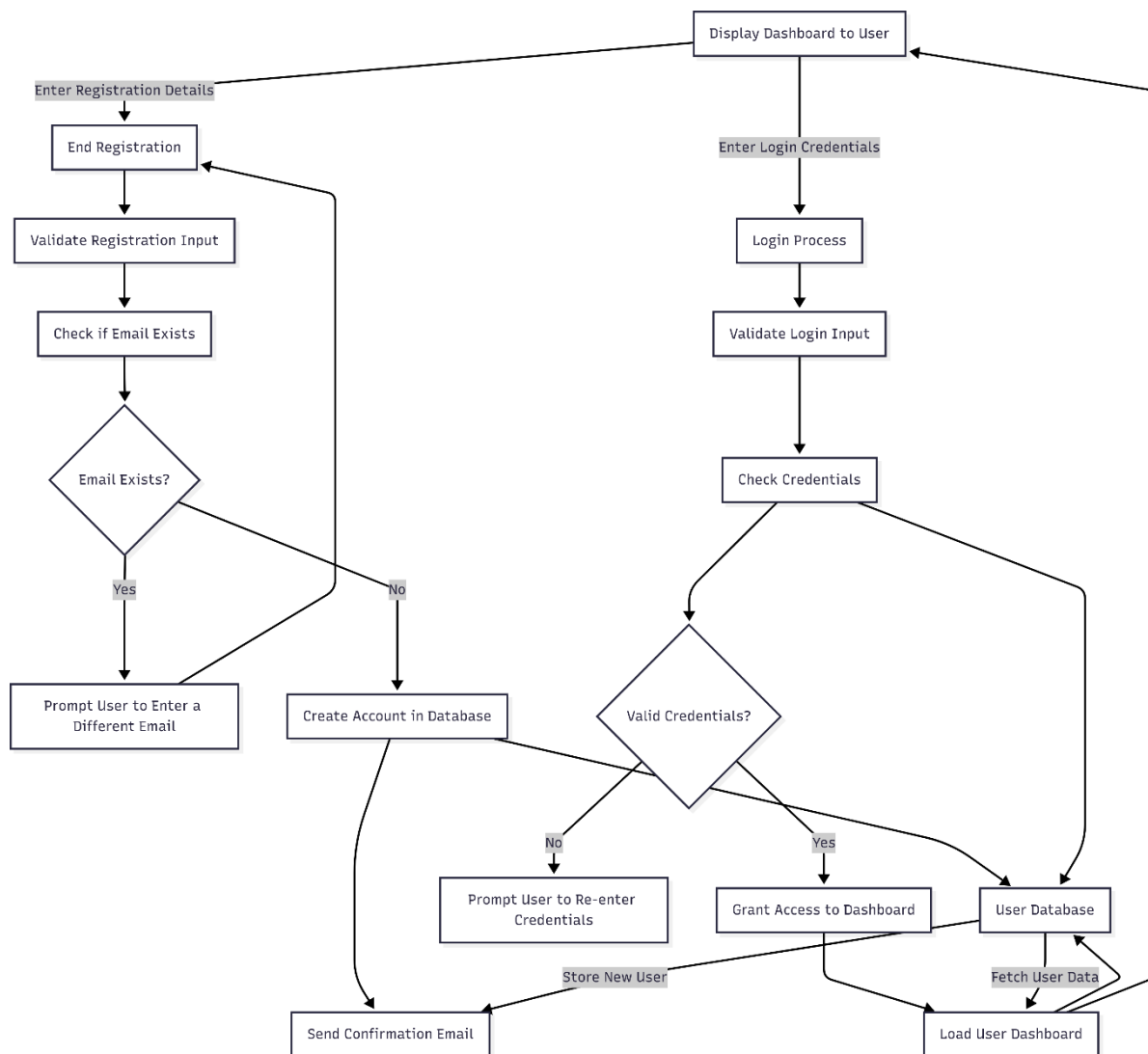


Fig 4.4.3 DFD Level-2 Diagram

Summary

The input design focuses on collecting accurate and meaningful data for the system. Well-designed input forms and interfaces should be simple, user-friendly, consistent, and efficient, ensuring data accuracy and completeness. The main objective is to create effective data entry methods, reduce unnecessary input, and apply validation techniques to maintain data reliability.

The output design deals with presenting processed information in a clear and useful manner. It ensures that outputs are relevant, well-structured, and delivered on time to the right users. The goal is to avoid information overload while supporting effective decision-making through clear and meaningful reports or displays.

In system modeling, UML diagrams help visualize system structure and behavior. The Use Case Diagram specifically represents the interaction between users (actors) and the system.

Chapter – 5

TECHNOLOGIES AND TOOLS

5.1 Programming Languages

Frameworks / Libraries

Various frameworks and libraries are essential for building an effective wildfire detection system. Tools like TensorFlow and PyTorch are widely used for developing and training deep learning models capable of detecting fire and smoke patterns. OpenCV is used for image and video processing, helping analyze visual data from cameras. Keras simplifies neural network implementation, especially for beginners. For web development and API creation, frameworks such as Flask and Django are commonly used. Additionally, libraries like NumPy and Pandas assist in handling and processing large datasets efficiently.

Database

Databases play a crucial role in storing and managing the large volume of data generated by wildfire detection systems. Relational databases such as MySQL and PostgreSQL are commonly used to store structured data like sensor readings, alerts, and logs. NoSQL databases like MongoDB are suitable for handling unstructured data such as images and large-scale sensor streams. Additionally, cloud-based platforms like Firebase provide real-time database capabilities and enable instant notifications, making them ideal for remote monitoring systems.

HARDWARE & SOFTWARE REQUIREMENTS

5.2 SOFTWARE REQUIREMENTS

Operating System:	Windows 7, 8, or 10
Frontend Technologies:	HTML, CSS, Bootstrap, and JavaScript
Programming Language:	Python
Libraries/Frameworks:	Flask, Pandas, PyTorch, Scikit-learn, Librosa, NumPy, Seaborn, Matplotlib
IDE/Development Tool:	Visual Studio Code (VS Code)
Server Environment:	XAMPP Server
Database:	MySQL

5.3 HARDWARE REQUIREMENTS

Processor:	Intel i3 or any equivalent processor
RAM:	Minimum 8 GB
Storage:	At least 128 GB of hard disk space
Keyboard:	Standard Windows-compatible keyboard
Mouse:	Two-button or three-button mouse
Monitor:	Any compatible display unit

Summary

The development of the wildfire detection system relies on various programming languages, frameworks, and libraries to ensure efficient implementation. Deep learning frameworks like TensorFlow and PyTorch are used for training models to detect fire and smoke. OpenCV supports image and video processing, while Keras simplifies neural network development. For web-based interfaces and APIs, Flask and Django are commonly used. Data handling is managed using libraries such as NumPy and Pandas.

Databases are essential for storing system data. MySQL and PostgreSQL handle structured data like logs and alerts, while MongoDB manages unstructured data such as images. Cloud platforms like Firebase enable real-time data storage and instant notifications.

The software requirements include a Windows operating system, Python programming language, frontend technologies (HTML, CSS, Bootstrap, JavaScript), and development tools like Visual Studio Code and XAMPP Server. Libraries such as Flask, Pandas, PyTorch, Scikit-learn, NumPy, Matplotlib, and Seaborn are used for development and analysis.

The hardware requirements are minimal, including an Intel i3 processor (or equivalent), at least 8 GB RAM, 128 GB storage, and basic input/output devices like a keyboard, mouse, and monitor.

Chapter – 6

IMPLEMENTATION

6.1 Data Collection

- **Objective:**

To acquire a well-structured dataset consisting of images and videos that include fire, smoke, and human presence for training detection models.

- **Details:**

Data is collected from multiple sources such as publicly available datasets, surveillance recordings, and simulated scenarios. This ensures diversity in terms of environment, lighting conditions, and background variations, improving the model's reliability and performance.

- **Dataset**

Split:

The collected dataset is divided into three parts: 70% for training, 15% for validation, and 15% for testing. This division supports proper model development, performance evaluation, and optimization.

Data Preprocessing

- **ImageProcessing:**

Prepare and standardize image data by correcting brightness, contrast, and color levels. Data augmentation methods such as rotation, scaling, and flipping are applied to increase variability and improve model generalization.

- **VideoProcessing:**

Process video inputs by extracting frames, reducing noise, and performing segmentation where necessary. The extracted frames are then converted into a uniform format and resolution to ensure compatibility with the training model.

6.2 Model Training

- **DetectionModels:**

The YOLOv8 and YOLOv9 algorithms are trained using the preprocessed image and video datasets to accurately identify fire, smoke, and human presence in various scenarios.

- **DataAugmentation:**

During the training phase, augmentation techniques are applied to enhance model performance and adaptability. These methods help the model handle variations in lighting conditions, viewing angles, and object sizes effectively.

Model Evaluation

- **Performance Metrics:** Assess detection models using metrics such as Intersection over Union (IoU), precision, recall, and F1-score to evaluate model accuracy and effectiveness in identifying fire, smoke, and humans.
- **Validation:** Use validation datasets to fine-tune models and ensure they generalize well to unseen data, refining them to reduce false positives and negatives in critical scenarios.

Model Saving

- **Model Serialization:** Save trained models in suitable formats (e.g., .pt for PyTorch) for deployment. Store model checkpoints to allow easy reloading and adjustments based on future data or requirements.

2. User

6.2.1 Register

- **Objective:** Users create an account to access the fire detection and alert system.
- **Details:** Registration includes providing credentials, contact information, and alert preferences to receive timely notifications in case of fire or smoke detection.

6.2.2 Login

- **Objective:** Registered users log in to access system features.
- **Details:** Manages user authentication and session control, ensuring secure access to personalized alert settings and detection data.

6.2.3 Input/Upload Image Feed

- **Objective:** Users upload video feeds or connect live camera feeds for real-time monitoring and detection.
- **Details:** Supports video or camera feed integration to monitor areas for fire, smoke, and human presence. Handles receiving and validating input data for processing by detection models.

6.2.4 View Detection Results

- **Objective:** Users view detection results for any identified risks.
- **Details:** Displays detected instances of fire, smoke, and human presence in real-time.

6.2.5 Logout

- **Objective:** Users log out to secure their session and personal data.
- **Details:** Manages session termination, ensuring that user data.

6.3 Algorithms Used

Forest wildfire and smoke detection systems utilize a combination of computer vision, machine learning, and deep learning techniques to identify fire-related events in real time. Earlier approaches rely on basic image processing methods such as color segmentation and thresholding, where fire-prone regions are detected based on color intensity in RGB or HSV spaces, particularly focusing on red and yellow tones. Motion-based techniques like background subtraction and optical flow are also applied to capture dynamic patterns, including flame flickering and smoke movement.

In recent years, deep learning methods have become the primary approach for detection tasks. Convolutional Neural Networks (CNNs) are commonly employed for extracting features and classifying images containing fire and smoke. Advanced object detection models, especially YOLO (You Only Look Once) variants like YOLOv8 and YOLOv9, are highly effective due to their ability to perform fast and accurate real-time detection. These models process images by dividing them into grids and predicting bounding boxes along with class labels, enabling quick identification of affected regions.

Other techniques, such as Region-Based CNN (R-CNN), Fast R-CNN, and Faster R-CNN, offer more precise detection but require higher computational resources. Smoke detection can also involve texture-based analysis and temporal variations, often supported by machine learning methods like Support Vector Machines (SVM) and Random Forest algorithms.

To improve system reliability, some approaches combine visual data with inputs from sensors such as temperature and gas detectors. Additionally, sequential models like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are used to analyze time-based patterns in video data. Overall, these combined techniques enable efficient, accurate, and robust detection of wildfires, even

under challenging environmental conditions.

6.4 Code Snippets

```
1 from flask import Flask, render_template, redirect, url_for, flash, request, session, jsonify, send_from_directory
2 from flask_sqlalchemy import SQLAlchemy
3 from flask_bcrypt import Bcrypt
4 from ultralytics import YOLO
5 from PIL import Image
6 import numpy as np
7 import cv2
8 import os
9 import imghdr
10 from datetime import datetime
11
12
13 app = Flask(__name__)
14 app.config['SECRET_KEY'] = 'your_secret_key'
15 app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
16 UPLOAD_FOLDER = 'uploads'
17 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
18 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
19
20 db = SQLAlchemy(app)
21 bcrypt = Bcrypt(app)
22
23 class User(db.Model):
24     id = db.Column(db.Integer, primary_key=True)
25     username = db.Column(db.String(150), unique=True, nullable=False)
26     email = db.Column(db.String(150), unique=True, nullable=False)
27     password = db.Column(db.String(150), nullable=False)
28     age = db.Column(db.Integer, nullable=False)
29     gender = db.Column(db.String(1), nullable=False)
```

Fig 6.4.1 Code Snippets

```
mobile = db.Column(db.String(15), nullable=False)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact')
def contact():
    return render_template('contact.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user = User.query.filter_by(email=email).first()
        if user and bcrypt.check_password_hash(user.password, password):
            session['user_id'] = user.id
            return redirect(url_for('home'))
        else:
            flash('Invalid email or password.', 'danger')
    return render_template('auth.html')
```

Fig 6.4.2 Code Snippets

```

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')
        confirm_password = request.form.get('confirm_password')
        age = request.form.get('age')
        gender = request.form.get('gender')
        mobile = request.form.get('mobile')

        if len(mobile) != 10 or not mobile.isdigit():
            flash('Mobile number must be exactly 10 digits.', 'danger')
            return render_template('auth.html')

        if User.query.filter_by(email=email).first():
            flash('Email address already in use. Please choose a different one.', 'danger')
            return render_template('auth.html')

        if password != confirm_password:
            flash('Passwords do not match.', 'danger')
            return render_template('auth.html')

        if len(password) < 8:
            flash('Password must be at least 8 characters long.', 'danger')
            return render_template('auth.html')

```

Fig 6.4.3 Code Snippets

```

def upload():
    if detected_classes:
        detection_message = "Classes Detected: " + ", ".join(detected_classes)
    else:
        detection_message = "No Target Class Detected"

    print(detection_message)

    return render_template('upload.html', result_image=url_for('uploaded_file', filename='result_' + uploaded_file.filename),
                           result_image_data=result_image_data)

    return render_template('upload.html')

@app.route('/uploads/<filename>')
def uploaded_file(filename):
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename)

@app.route('/logout')
def logout():
    session.pop('user_id', None)
    flash('You have been logged out.', 'info')
    return redirect(url_for('login'))

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)

```

Fig 6.4.4 Code Snippets

Summary

This section describes the complete workflow of the wildfire detection system, from data collection to model deployment and user interaction.

The system begins with data collection, where images and videos of fire, smoke, and human presence are gathered from multiple sources such as public datasets, surveillance footage, and simulated environments. The dataset is divided into training (70%), validation (15%), and testing (15%) to ensure proper model development and evaluation.

In the data preprocessing stage, images are enhanced through brightness and contrast adjustments, while augmentation techniques like rotation and scaling improve model robustness. Video data is processed by extracting frames and standardizing them for training.

During model training, advanced deep learning algorithms such as YOLOv8 and YOLOv9 are used to detect fire, smoke, and human presence. The model's performance is evaluated using metrics like precision, recall, F1-score, and IoU, and further refined using validation datasets. Trained models are then saved for deployment.

Chapter – 7

SYSTEM STUDY AND TESTING

7.1 Feasibility Study

In this phase, the practicality of the project is evaluated, and a basic proposal is prepared outlining a general plan along with estimated costs. During system analysis, a feasibility study is conducted to determine whether the proposed system can be successfully implemented without placing unnecessary strain on the organization. A clear understanding of the system's major requirements is essential for this evaluation.

The feasibility analysis mainly focuses on three important aspects:

- **Economic Feasibility:**
Examines the cost-effectiveness of the project, ensuring that the expected benefits justify the investment.
- **Technical Feasibility:**
Assesses whether the required technology, tools, and expertise are available to develop and implement the system.
- **Social Feasibility:**
Evaluates how well the system will be accepted by users and its overall impact on society or the organization.

Economical Feasibility

This aspect evaluates the financial impact of the proposed system on the organization. Since the budget allocated for research and development is limited, it is important to ensure that all expenses are reasonable and justified.

The system is designed to remain within the available budget by utilizing mostly open-source and freely available technologies. Only a few specialized or customized components require additional investment. As a result, the overall cost of development is controlled, making the project economically viable.

Technical Feasibility

This aspect focuses on evaluating whether the proposed system can be developed and implemented using

the available technical resources. The system should not place excessive demands on existing hardware, software, or technical expertise, as this could increase complexity and cost for the organization.

The proposed solution is designed to operate with minimal resource requirements, ensuring easy implementation. It does not require significant modifications to the current infrastructure, making it technically practical and feasible.

Social Feasibility

This aspect evaluates how well the proposed system will be accepted by its users. It involves ensuring that users are properly trained and able to operate the system with confidence and ease. The system should be designed in a way that users perceive it as helpful rather than intimidating.

User acceptance largely depends on effective training methods and awareness programs that familiarize them with the system's features and benefits. Building user confidence is essential, as it encourages them to use the system efficiently and provide valuable feedback. Such feedback is important for improving the system, since end users play a key role in its successful adoption.

System Testing

The main objective of testing is to identify errors and weaknesses in the system. It involves examining the software to detect possible faults and ensure that all components function as expected. Testing evaluates individual modules, integrated units, and the complete system to verify their performance.

This process ensures that the software meets specified requirements and satisfies user expectations without causing unacceptable failures. Different types of testing are carried out, each focusing on specific aspects of the system to ensure overall quality and reliability.

7.2 Types of Tests

Unit testing Unit testing focuses on verifying the correctness of individual components or modules within a software application. It involves creating test cases to ensure that the internal logic of the program works as intended and that given inputs produce the expected outputs.

During this phase, all decision paths and code flows are examined to confirm proper functionality. Unit testing is performed after the development of each module and before integrating it with other parts of the system. It is considered a structural testing method, as it requires knowledge of the internal code.

These tests operate at the component level, checking specific functions, business processes, or configurations. The goal is to ensure that every possible execution path behaves according to the defined specifications, with clearly identified inputs and expected results.

Integration testing

Integration testing is performed to verify that multiple software modules function correctly when combined into a single system. Even if individual components pass unit testing, this phase ensures that their interaction is consistent and error-free.

This type of testing focuses on identifying issues that may arise due to improper communication between modules, such as interface defects or data mismatches. It is typically carried out in an incremental manner, where components are gradually integrated and tested together on a common platform.

The primary objective is to confirm that all integrated parts of the system work together smoothly and produce the expected outcomes.

Test Results: All defined test cases were executed successfully, and no defects were identified.

Acceptance Testing

Acceptance testing is the final stage of testing, where the system is evaluated by end users to ensure it meets their requirements and expectations. This phase requires active user involvement to validate that the system performs its intended functions correctly in a real-world environment.

It confirms that the developed system aligns with the specified functional requirements and is ready for deployment.

Test Results: All test cases were completed successfully, and no issues were found.

Functional testing

Functional testing is conducted to verify that the system's features operate according to the specified business and technical requirements, as well as system documentation and user guidelines. It ensures that each function of the application performs as expected.

This type of testing focuses on several key aspects:

- **Valid Input:** The system should correctly accept and process all defined valid inputs.
- **Invalid Input:** Inputs that do not meet the specified criteria must be rejected appropriately.
- **Functions:** All identified system functions should be executed and validated.
- **Output:** The generated outputs must match the expected results for different input conditions.
- **System/Procedures:** Interactions with other systems or procedures should work without errors.

The planning and execution of functional testing are based on system requirements, critical functionalities, and specific test scenarios. It also involves ensuring complete coverage of business workflows, data fields, and sequential processes. As testing progresses, additional test cases may be introduced, and the effectiveness of existing tests is continuously evaluated. White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black box testing is a method of evaluating software without examining its internal code, structure, or implementation details. In this approach, the system is treated as a “black box,” where the tester focuses only on providing inputs and analyzing the resulting outputs.

Test cases are designed based on requirement specifications or functional documents rather than the internal logic of the system. The primary goal is to ensure that the software behaves as expected for different input conditions.

Test objectives

- Ensure that all input fields function correctly.
- Verify that navigation between pages works through the specified links.
- Confirm that system responses, messages, and screens are displayed without delay

Features to be tested

- Validate that all inputs follow the required format.
- Ensure that duplicate entries are not accepted.
- Check that all links direct users to the appropriate pages.

Summary

The feasibility study evaluates whether the wildfire detection system can be successfully implemented. It focuses on three key aspects:

- **Economic Feasibility:** Ensures the project is cost-effective by using mostly open-source tools, keeping development within budget.
- **Technical Feasibility:** Confirms that the system can be developed using available technologies and resources without requiring major infrastructure changes.
- **Social Feasibility:** Assesses user acceptance, emphasizing ease of use, proper training, and user confidence for successful adoption.

Overall, the system is found to be practical, affordable, and user-friendly, making it feasible for real-world deployment.

Chapter – 8

RESULTS AND DISCUSSION

8.1 Output Screens:

Home Page: ForestSync AI is an advanced wildfire and smoke detection platform that leverages cutting-edge artificial intelligence for instant monitoring and early warning systems. The platform empowers conservation efforts through precise real-time monitoring, predictive analytics, and coordinated emergency response capabilities to protect forests and communities from fire threats.

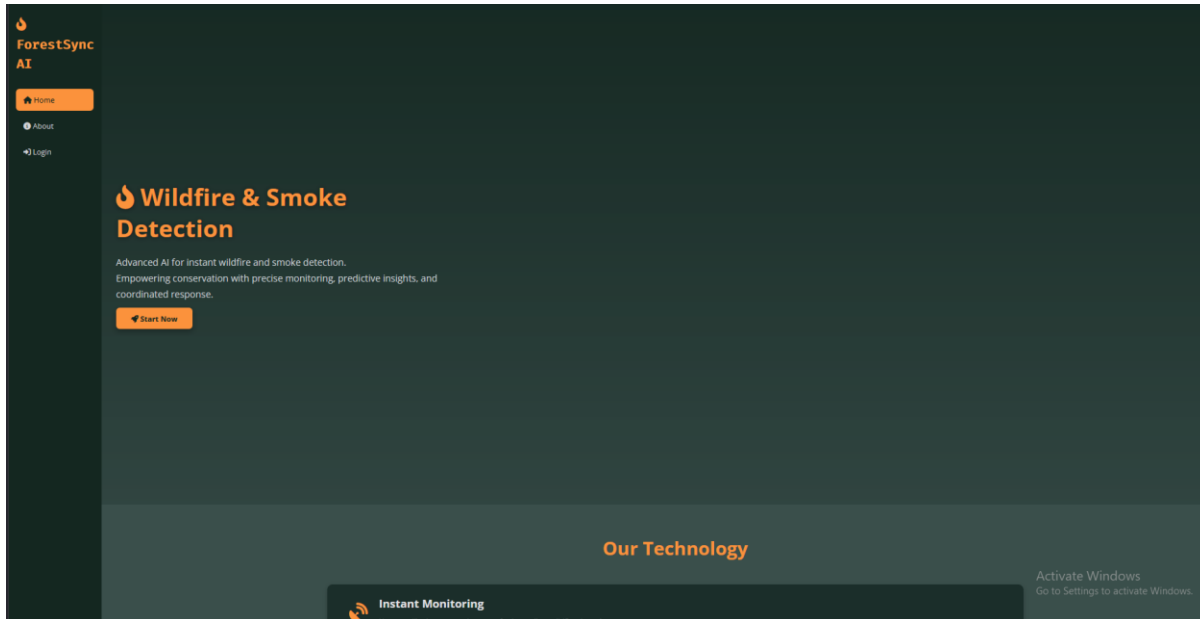


Fig 8.1.1 Home Page

About Us Page: ForestSync AI harnesses cutting-edge machine learning technology to revolutionize wildfire and smoke detection through satellite imagery and advanced sensor networks. Our intelligent platform delivers instant monitoring capabilities and predictive analytics, empowering conservationists, firefighters, and environmental agencies with the tools needed for rapid response and effective forest protection.

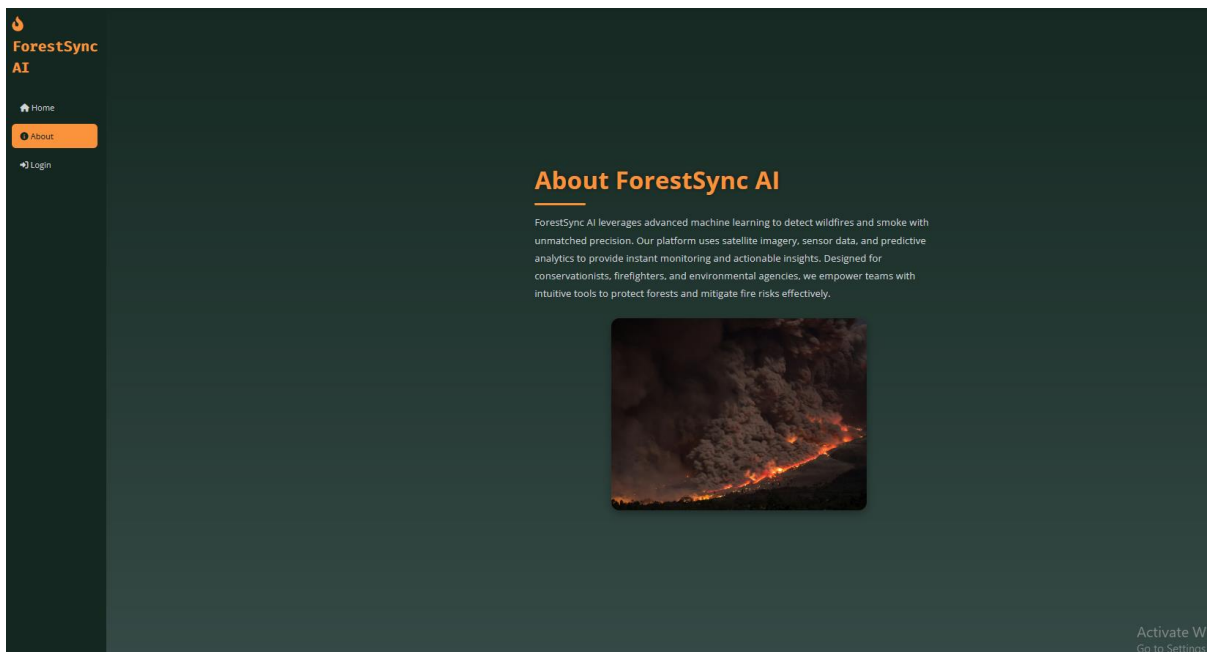


Fig 8.1.2 About Us Page

Login Page: Access your ForestSync AI dashboard with secure login credentials to monitor real-time wildfire detection data and alerts. Join our network of environmental professionals dedicated to protecting forests through advanced AI-powered surveillance and early warning systems.

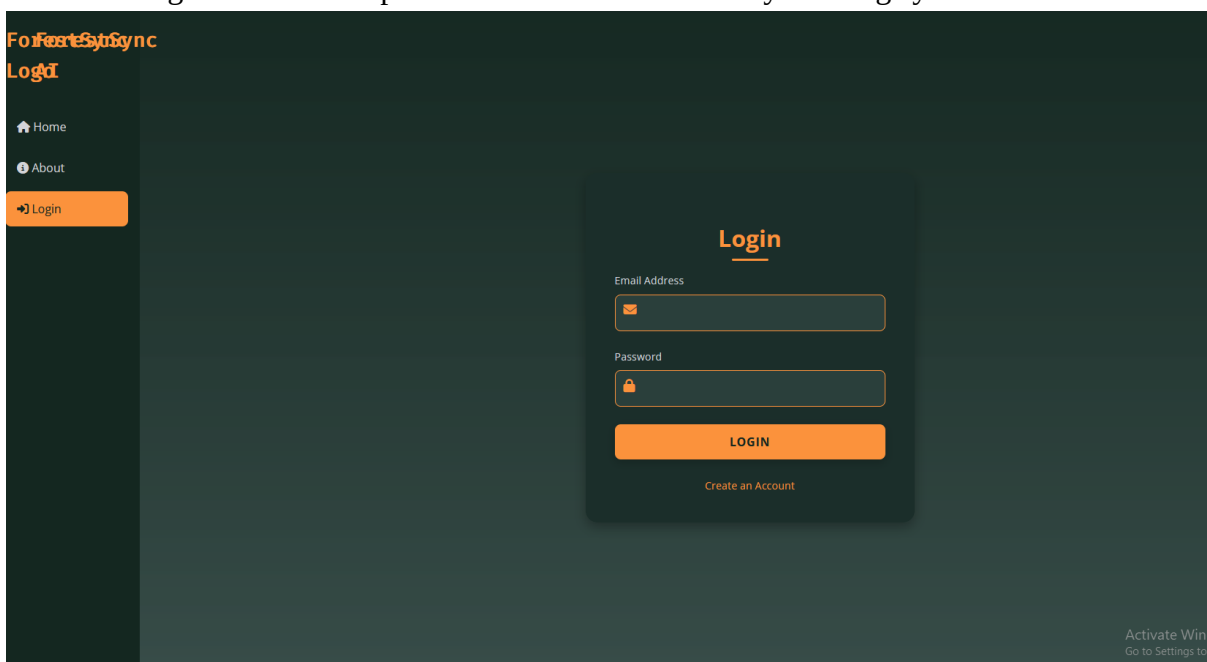


Fig 8.1.3 Login Page

Register Page: Create your ForestSync AI account to join the frontline of wildfire prevention and forest conservation efforts worldwide. Register now to access cutting-edge detection technology, real-time alerts, and collaborate with environmental professionals protecting our natural ecosystems.

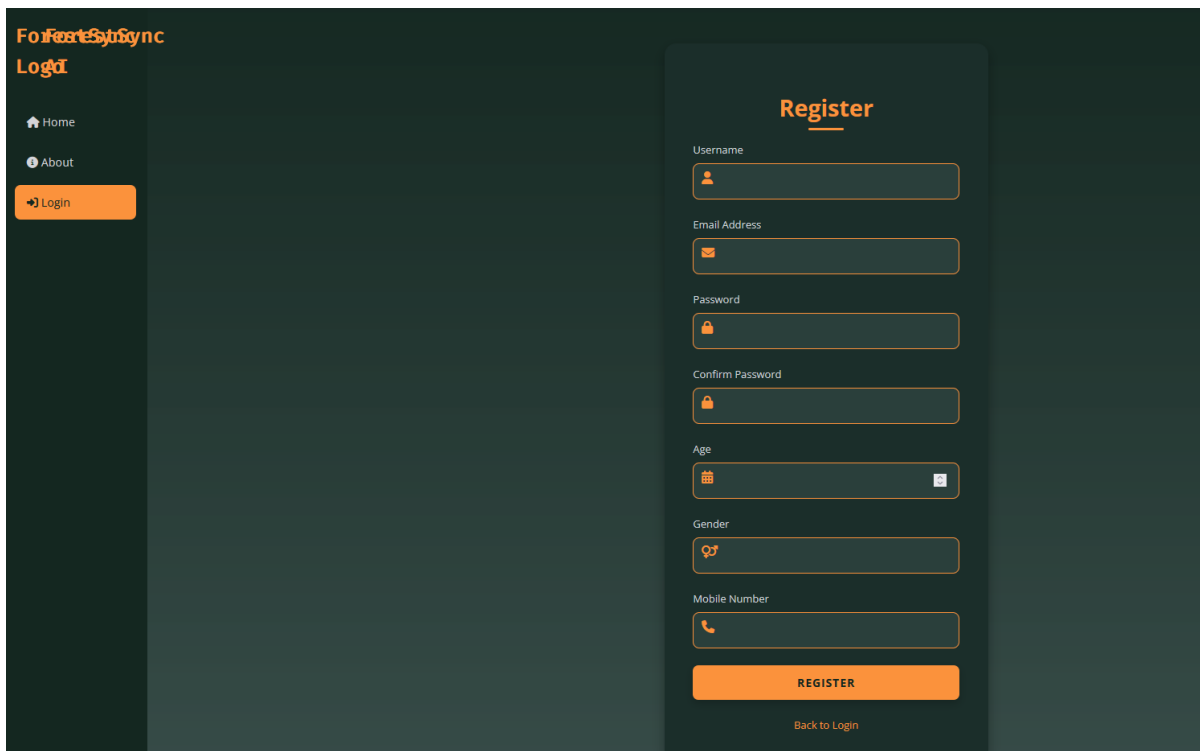
The image shows the 'Register' page of the ForestSync AI application. On the left, there is a dark sidebar with the 'ForestSync AI' logo at the top, followed by 'Login' and a 'Logout' button. The main content area has a dark background. At the top center, the word 'Register' is displayed in a light blue font. Below it, there are several input fields: 'Username' with a person icon, 'Email Address' with an envelope icon, 'Password' with a lock icon, 'Confirm Password' with a lock icon, 'Age' with a calendar icon, 'Gender' with a male/female icon, and 'Mobile Number' with a phone icon. At the bottom of the form is a large blue 'REGISTER' button, and below that is a smaller link that says 'Back to Login'.

Fig 8.1.4 Register Page

Home Page: Welcome to ForestSync AI's main dashboard, where cutting-edge artificial intelligence meets forest conservation to deliver instant wildfire and smoke detection capabilities. Experience the power of advanced technology that protects our forests through real-time monitoring, risk forecasting, and coordinated emergency response systems.

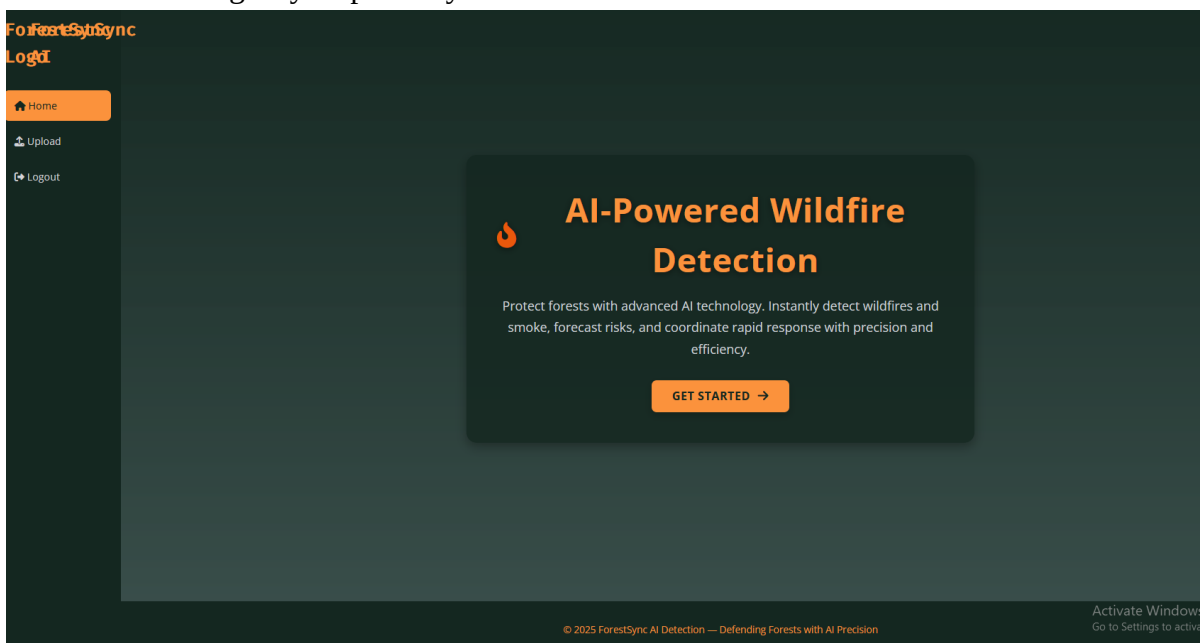


Fig 8.1.5 Home Page

Prediction Page: Upload your forest imagery to ForestSync AI's advanced wildfire analysis system for instant detection and assessment of fire risks and smoke patterns. Our sophisticated machine learning algorithms will analyze your images in real-time, providing critical insights for forest management and

emergency response planning.

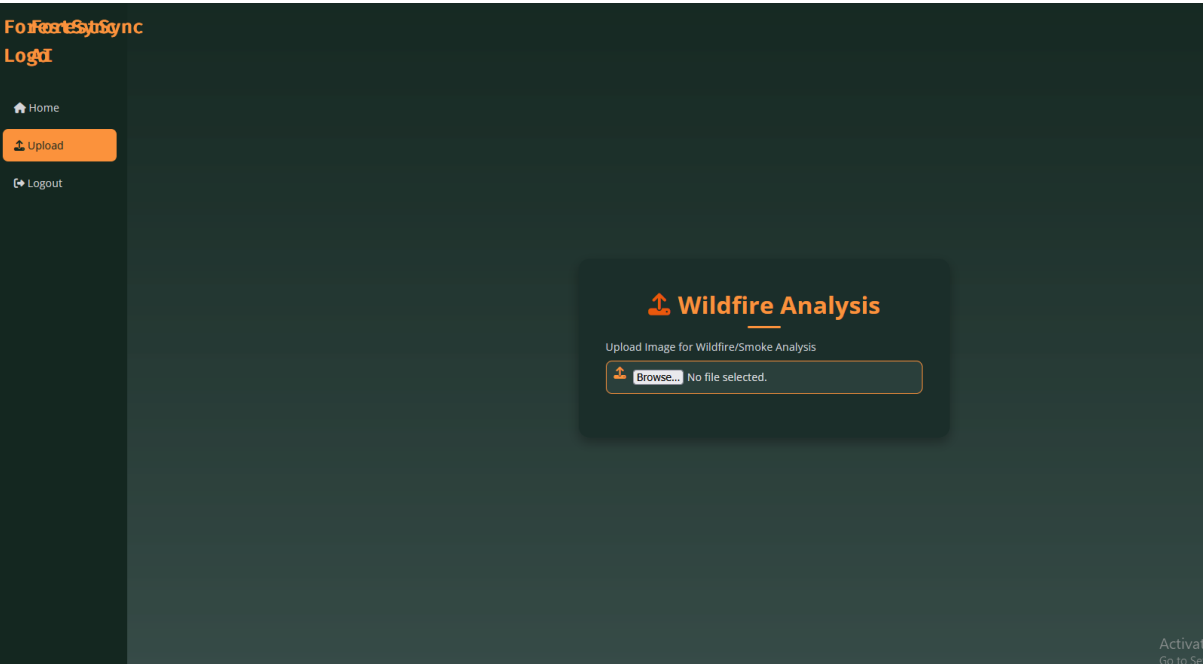


Fig 8.1.6 Prediction Page

Output Page: ForestSync AI has successfully detected and classified both smoke and fire in the uploaded image, demonstrating real-time analysis capabilities with precise bounding box identification. The system provides instant visual feedback with labeled detection zones, enabling rapid assessment and response coordination for fire emergency situations.

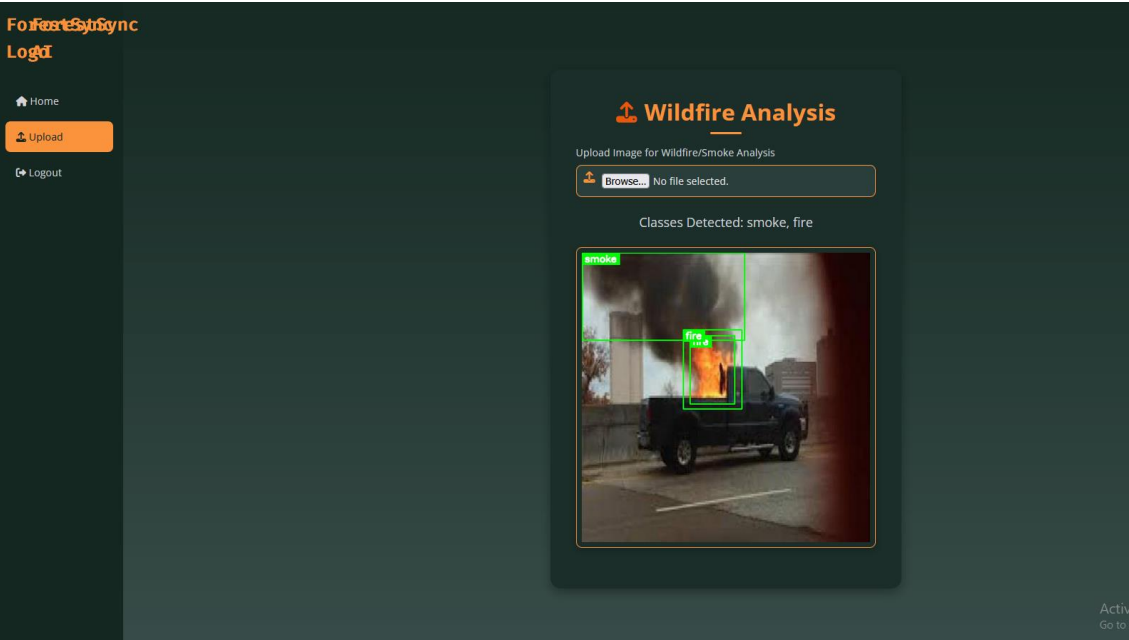
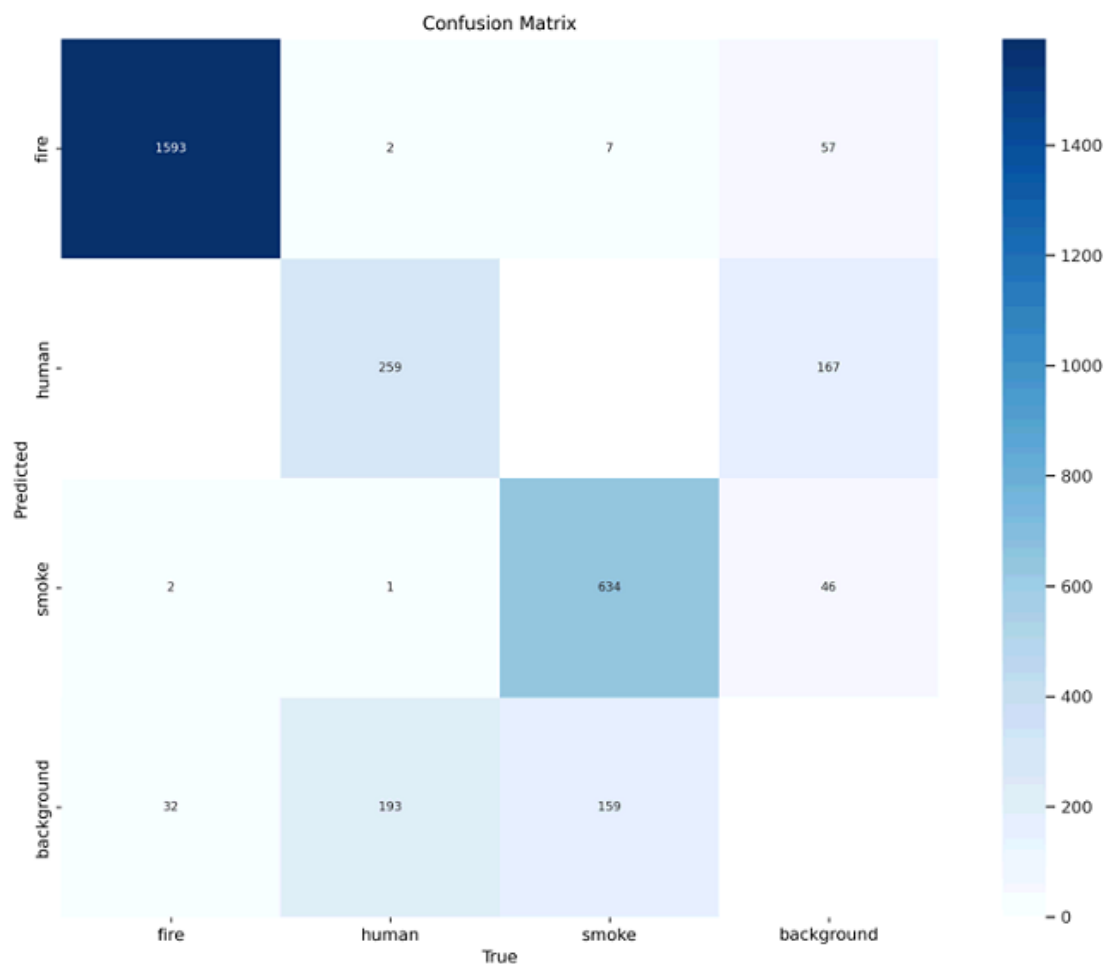


Fig 8.1.7 Output Page

8.2 Result Analysis

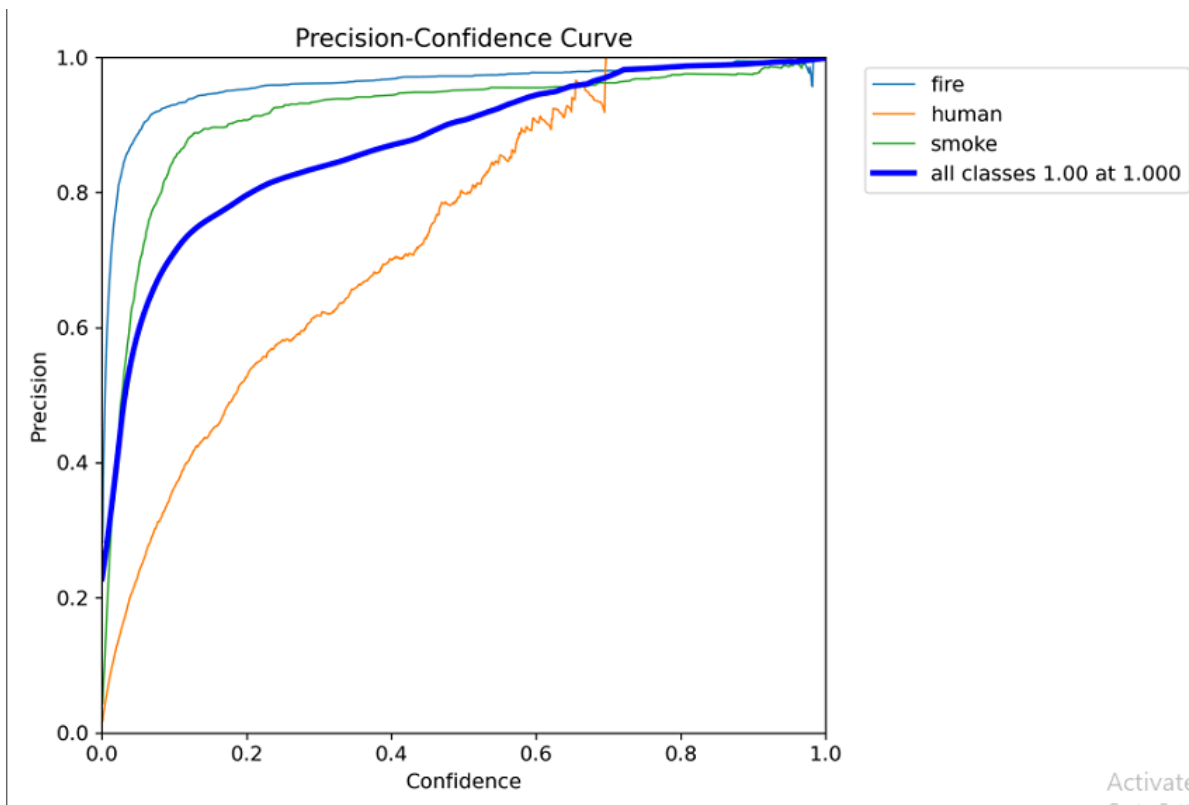
YOLOv9:



The confusion matrix showcases the performance of the YOLOv9 model in distinguishing between the four classes: fire, human, smoke, and background.

- **Fire:** The model correctly classified 1593 instances as fire, with minimal misclassifications. Only 57 instances were misclassified as background, and 7 as smoke, indicating excellent accuracy for this class.
- **Human:** The model detected 259 humans correctly, but 167 instances were misclassified as smoke and 193 as background. This highlights the model's difficulty in distinguishing humans from other objects, particularly smoke.

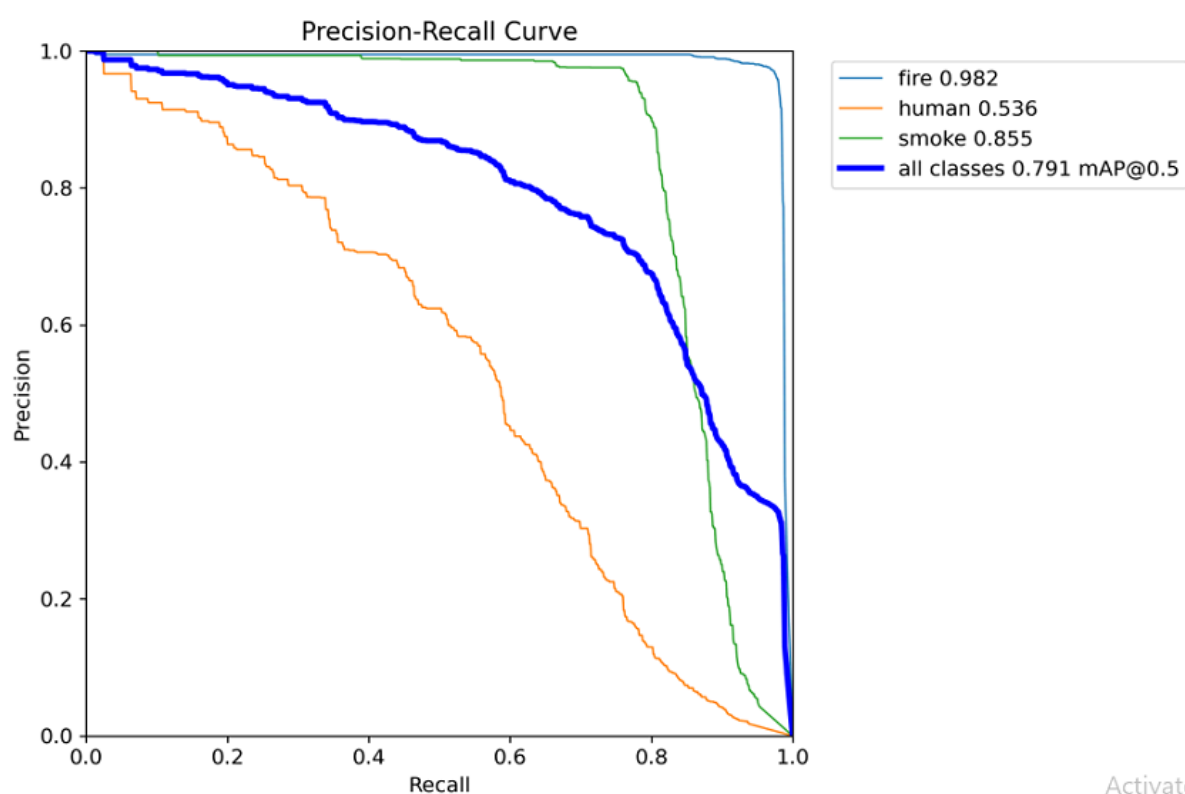
- **Smoke:** With 634 correct predictions, the model performs well here, but some instances were classified as fire (2) or background (46).
- **Background:** Background detection is mixed; 159 instances were confused with smoke, and 32 with fire, which might indicate overlapping visual features. Improving feature extraction or class separation could address these issues.



The precision-confidence curve visualizes the relationship between confidence scores and precision for each class:

- **Fire:** Maintains high precision across all confidence levels, showing that the model is highly reliable in detecting fire even at lower thresholds.
- **Human:** Precision drops significantly at lower confidence levels, meaning false positives are common for this class. This indicates either overlapping features with other classes or insufficient training samples for human detection.

- **Smoke:** Performs better than "human" but less consistently than "fire." Precision improves significantly with higher confidence, suggesting that strict thresholds can reduce false positives.
- **Overall Performance:** The combined "all classes" curve confirms that the model achieves high overall precision at high confidence levels but struggles to balance precision and recall for certain classes like humans.



Activate
Caption

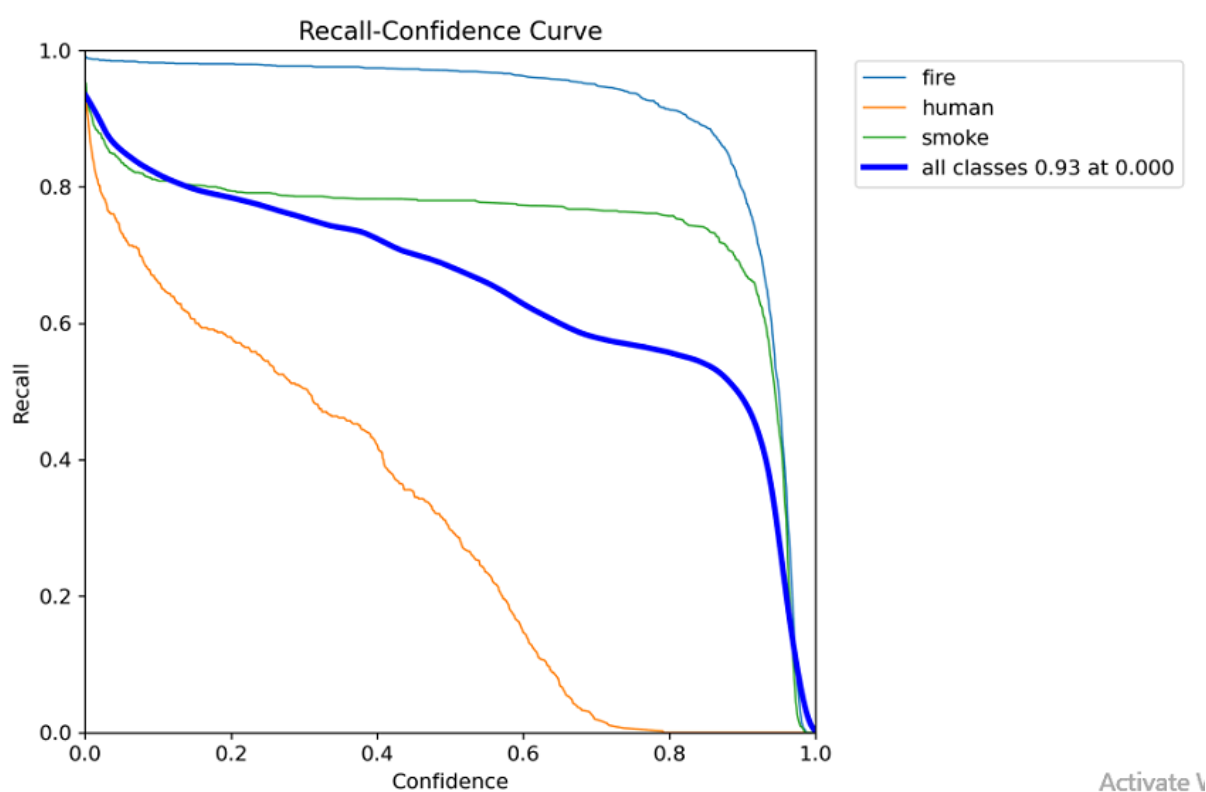
Precision-Recall

Curve:

The precision-recall curve provides a comprehensive view of the model's trade-offs between precision and recall for all classes.

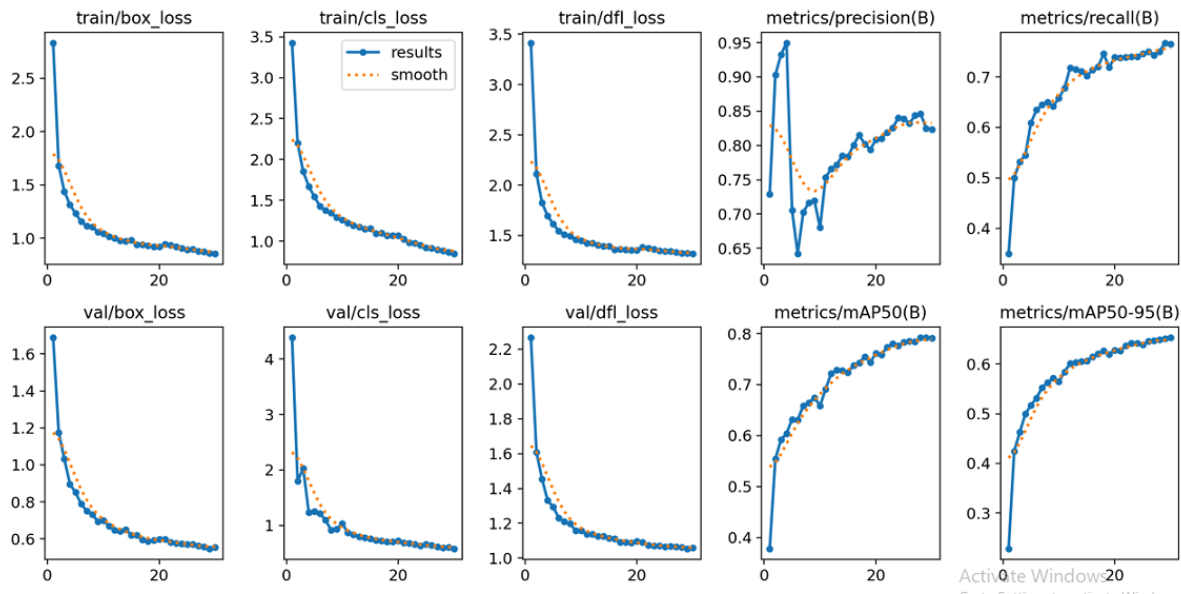
- **Fire (0.982):** This class has an almost perfect precision-recall balance, meaning the model is both accurate and exhaustive in identifying fire instances.

- **Human (0.536):** The lowest-performing class, with precision and recall decreasing significantly as recall increases. This indicates the model struggles to capture all human instances, likely due to insufficient features or overlap with the background.
- **Smoke (0.855):** Shows a strong balance between precision and recall, although there is room for improvement, particularly in reducing false negatives.
- **Mean Average Precision (mAP@0.5 = 0.791):** While overall performance is commendable, the relatively low mAP indicates that the model struggles to generalize well for all classes, particularly humans. Enhancing the dataset, using data augmentation, or fine-tuning the model could improve this score further.



- **Fire:** High recall across all confidence levels, meaning fire detection is very reliable.
- **Smoke:** Moderate recall drop at higher confidence levels.

- **Human:** Steep drop, indicating the model struggles with human detection at higher confidence thresholds.
- **All Classes:** Combined performance is strong overall.



- **Training Losses:** The losses for bounding boxes, classification, and localization (top row) decrease steadily, indicating the model is learning effectively.
- **Validation Losses:** Similar trends for the validation set (bottom row) show that the model generalizes well.
- **Metrics:**
 - **Precision & Recall:** Both improve during training, showing the model's ability to make accurate predictions.
 - **mAP (Mean Average Precision):** Gradual improvement reflects better overall detection performance.



This illustrates how the model identifies fire, smoke, and human presence in test images using bounding boxes.

- Fire detection is highly accurate and consistent across most cases.
- Smoke detection performs fairly well but may face difficulties in complex or cluttered environments.
- Human detection shows comparatively lower accuracy, possibly due to occlusion or challenging visual conditions.

Summary

The system interface of ForestSync AI includes multiple user-friendly screens designed for smooth interaction and real-time monitoring. The Home Page introduces the platform and its AI-based wildfire

detection capabilities. The About Us Page explains the system's purpose and technology. The Login and Register Pages allow secure user access, while the Dashboard (Home) provides real-time monitoring features. The Prediction Page enables users to upload images for analysis, and the Output Page displays detection results with bounding boxes highlighting fire, smoke, and human presence.

In the result analysis, the YOLOv9 model demonstrates strong performance, especially in detecting fire and smoke:

- Fire Detection: Highly accurate with very few misclassifications and excellent precision-recall balance.
- Smoke Detection: Performs well with good accuracy, though minor confusion with background exists.
- Human Detection: Shows lower accuracy due to overlap with smoke and background, indicating a need for improved training data.
- Background Classification: Some misclassification occurs, suggesting overlapping visual features.

Performance metrics indicate:

- High precision at higher confidence levels, especially for fire detection.
- Precision-Recall trade-off shows strong results for fire (0.982), moderate for smoke (0.855), and weaker for humans (0.536).
- Overall $mAP@0.5 = 0.791$, indicating good but improvable model performance.

Training and validation losses decrease steadily, confirming effective learning and good generalization. Visual outputs using bounding boxes show that fire detection is highly reliable, smoke detection is reasonably accurate, and human detection requires improvement.

Chapter- 9

CONCLUSION AND FUTURE WORK

9.1 CONCLUSION:

This paper presented a deep learning-based solution for real-time wildfire and smoke detection using YOLOv8 and YOLOv9. Both models demonstrated effective performance, with YOLOv9 providing superior precision and recall compared to YOLOv8, particularly for small and occluded objects like smoke plumes. The proposed system processes visual data from diverse sources such as drones, satellites, and surveillance cameras, offering an efficient and scalable solution for large-scale wildfire monitoring. The results showed that YOLOv9's advanced architecture enhances detection accuracy, enabling quicker response times and more accurate wildfire management. This method has the potential to save lives, protect ecosystems, and reduce the devastating impact of wildfires.

9.2 ACHIEVEMENTS OF THE PROJECT:

Forest wildfire and smoke detection systems, although highly useful, have several important limitations. Environmental conditions such as fog, rain, clouds, and varying sunlight can significantly affect detection accuracy, often leading to false alarms or missed detections. These systems may confuse smoke with dust or mist, and small or early-stage fires can go unnoticed due to low visibility. In addition, the performance of AI-based models depends heavily on the availability of large and diverse datasets, which are often limited, reducing their ability to generalize across different forest environments. Hardware and infrastructure requirements, including cameras, drones, and sensors, can be expensive and difficult to maintain, especially in remote areas with limited power and connectivity. Furthermore, real-time processing demands high computational resources, making large-scale deployment challenging. Coverage is also restricted, as dense vegetation can block visibility, and satellite systems may not detect small fires due to resolution limitations. Overall, these challenges highlight the need for more robust, scalable, and accurate wildfire detection solutions.

9.3 LIMITATIONS:

Forest wildfire and smoke detection systems, although highly useful, have several important limitations. Environmental conditions such as fog, rain, clouds, and varying sunlight can significantly affect detection accuracy, often leading to false alarms or missed detections. These systems may confuse smoke with dust or mist, and small or early-stage fires can go unnoticed due to low visibility. In addition, the performance of AI-based models depends heavily on the availability of large and diverse datasets, which are often

limited, reducing their ability to generalize across different forest environments. Hardware and infrastructure requirements, including cameras, drones, and sensors, can be expensive and difficult to maintain, especially in remote areas with limited power and connectivity. Furthermore, real-time processing demands high computational resources, making large-scale deployment challenging. Coverage is also restricted, as dense vegetation can block visibility, and satellite systems may not detect small fires due to resolution limitations. Overall, these challenges highlight the need for more robust, scalable, and accurate wildfire detection solutions.

9.4 FUTURE ENHANCEMENT:

Future enhancements for the YOLOv9 model in wildfire, smoke, and human detection can focus on several key areas. First, improving the **human detection** performance is critical, as the model currently struggles with false positives and overlap with other classes. This can be achieved by incorporating **additional training data** with more varied human poses, lighting conditions, and occlusions, along with advanced data augmentation techniques to create a more robust model. Second, enhancing the **feature extraction** process could help in distinguishing between smoke and background, as the current confusion in detection suggests visual similarities. Third, implementing **multi-scale detection** could improve the model's performance in identifying smaller objects like smoke at different scales. Additionally, **fine-tuning the model** by using more diverse datasets and experimenting with different architectures can further reduce false negatives, especially in complex environments. Lastly, integrating **real-time learning** and **feedback loops** could enhance model adaptation in dynamic wildfire scenarios.

REFERENCES

1. Zaher, A., Al-Faqsh, A., Abdulredha, H., Al-Qudaihi, H., & Toaube, M. (2021). A Fire Prevention/Monitoring Smart System. 2021 2nd International Conference On Smart Cities, Automation & Intelligent Computing Systems (ICON-SONICS). IEEE.
2. Seetharaman, R., Sreeja, R. R., Dakshin, S. Vidhul, Nivetha, N., Gowsigan, S., & Barath, M. (2020). Analysis of a Real Time Fire Detection and Intimation System. IEEE.
3. Ahmad Azmil, M. S., Ya'acob, N., Tahar, K. N., & Sarnin, S. S. (2015). Wireless fire detection monitoring system for fire and rescue application. 2015 IEEE 11th International Colloquium on Signal Processing & Its Applications (CSPA). IEEE.
4. Luo, W. (2021). Research on fire detection based on YOLOv5. IEEE.
5. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, You Only Look Once: Unified, Real-Time Object Detection, in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016.
6. R. Girshick, Fast R-CNN, in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015.
7. S. Ren, K. He, R. Girshick, and J. Sun, Faster R-CNN, in *Advances in Neural Information Processing Systems*, 2015.
8. C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.
9. I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
10. R. Seetharaman, R. R. Sreeja, S. V. Dakshin, and N. S. Nithin, "Forest Fire Detection Using Deep Learning Techniques," in *Proc. IEEE Int. Conf. Communication and Electronics Systems (ICCES)*, 2020.

APPENDIX

A. Plagiarism Report

21% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **127 Not Cited or Quoted 21%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 14%  Internet sources
- 7%  Publications
- 12%  Submitted works (Student Papers)

B. Time and cost analysis of your project work

The time and cost analysis provides an estimate of the effort required to complete each phase of the project along with the actual time spent. This helps in understanding project efficiency, resource utilization, and deviations from the initial plan.

S.No	Activity	Estimated Time (Hours)	Actual Time Taken (Hours)	Estimated Cost (₹200/hr)
1	Requirement Gathering and Planning	10	12	2000
2	Literature Survey and Research	15	18	3000
3	Dataset Collection and Preprocessing	12	14	2400
4	System Design (Architecture + Flow)	10	12	2000
5	Frontend Development	30	35	6000
6	Backend Development	25	28	5000
7	Database Setup	8	10	1600
8	API Integration	8	10	1600
9	Bluetooth Integration	12	15	2400
10	AI/ML Model	15	17	3000

	Development			
11	Testing	20	22	4000
12	Deployment	6	7	1200
13	Report Writing &Documentation	12	12	12
Total	183 Hours	215 Hours	₹36,600	₹43,000

C. Conference Paper

FOREST WILD FIRE AND SMOKE DETECTION

1. Kuruva Rakshitha 2. Dr.K.Muni Sankar Assistant Professor
Department of Computer Sciences, Mohan Babu University, India.
E-mail: rakshitha970406@gmail.com , munisankar.k@mbu.asia

Abstract— Forest wildfires pose a significant threat to both the environment and human safety. Early detection is critical for minimizing damage and enabling timely interventions. This paper presents an automated wildfire and smoke detection system leveraging deep learning algorithms, specifically YOLOv9 and YOLOv8, to identify the presence of smoke and fire in forested areas. The proposed methodology uses image and video feeds from surveillance cameras, drones, and satellites to detect fire and smoke. YOLOv9 and YOLOv8 are employed for their efficiency in object detection, trained on a diverse dataset containing images of wildfire and smoke under various environmental conditions. The models are optimized to handle challenges such as varying lighting, weather, and occlusions from trees and vegetation. Results show that YOLOv8 achieves a mean Average Precision (mAP50) of 0.807, with high precision for detecting fire (0.959) and smoke (0.921). YOLOv9 also delivers solid performance, with a precision of 0.82366 and an mAP50-95 of 0.65323. The system provides an efficient solution for early wildfire detection, reducing response time, enhancing safety, and enabling timely resource deployment. This method can be integrated into existing monitoring systems for proactive wildfire management.

Keywords: Forest wildfires, wildfire detection, smoke detection, deep learning, YOLOv9, YOLOv8, object detection, surveillance cameras, drones, satellites, automated system.

I. Introduction

Forest wildfires represent a critical environmental and safety threat globally, destroying vast areas of forestland, wildlife habitats, and human settlements. The speed at which wildfires spread, coupled with their unpredictability, makes them extremely difficult to control and manage. A primary challenge in wildfire management is the timely detection of fires and smoke. Traditionally, wildfire detection methods rely on human patrols, satellite imagery [1], and manual surveillance, which are often slow and inefficient. As the frequency and intensity of wildfires continue to increase, there is a pressing need for more effective, automated solutions that can quickly identify early signs of wildfires and smoke.

Objective of the Study

This study focuses on developing an automated system to detect forest wildfires and smoke using deep learning techniques. The main objective is to design a real-time detection

system using YOLOv9 and YOLOv8 algorithms, which will significantly improve the speed and accuracy of wildfire detection. The system processes images and video data collected from surveillance cameras, drones, and satellite imagery to quickly identify smoke and fire in large forested regions. By leveraging the power of deep learning, this system aims to reduce the response time to emergencies, enhancing overall wildfire management efforts.

Novelty of the Approach

The novelty of this project lies in the use of advanced object detection algorithms—YOLOv9 and YOLOv8—for the automated detection of smoke and wildfire, which have not been extensively applied in this domain. These models are known for their ability to process images in real-time with high accuracy and speed, making them ideal for the dynamic and challenging environments typically encountered in wildfire detection. Unlike traditional methods, this approach eliminates human error and inefficiency, enabling continuous monitoring of vast forested areas and providing faster detection and response capabilities. Additionally, this project incorporates a diverse dataset containing wildfire images captured under various environmental conditions, ensuring that the system can detect smoke and fire even in challenging scenarios such as varying lighting, occlusions, and different vegetation types.

Significance of Early Wildfire Detection

Timely detection of wildfires is crucial in preventing catastrophic damage to natural ecosystems, human lives, and properties. The early identification of smoke and fire allows for prompt mobilization of firefighting resources, evacuation plans, and other protective measures. However, traditional detection systems often suffer from delays, limited coverage, and high

operational costs. This research proposes a solution to these challenges by leveraging the capabilities of deep learning and computer vision, specifically tailored to automatically detect and classify wildfires and smoke, ensuring quicker intervention and more effective resource allocation.

Scope of the Study

The scope of this study includes the development of an automated system capable of detecting wildfires and smoke from images and videos. The system utilizes YOLOv9 and YOLOv8 algorithms, which are trained on a comprehensive dataset of wildfire and smoke images under various environmental conditions. The system aims to:

- **Collect and preprocess data:** Gathering and preparing images and video feeds from diverse sources, such as satellites, drones, and surveillance cameras.
- **Train detection models:** Applying YOLOv9 and YOLOv8 models to detect smoke and fire in real-time.
- **Deploy the solution:** Implementing the system for large-scale coverage of forested areas, ensuring scalability and adaptability for various terrains.
- **Optimize system performance:** Focusing on enhancing detection accuracy and speed, reducing the time required to identify and respond to wildfires.

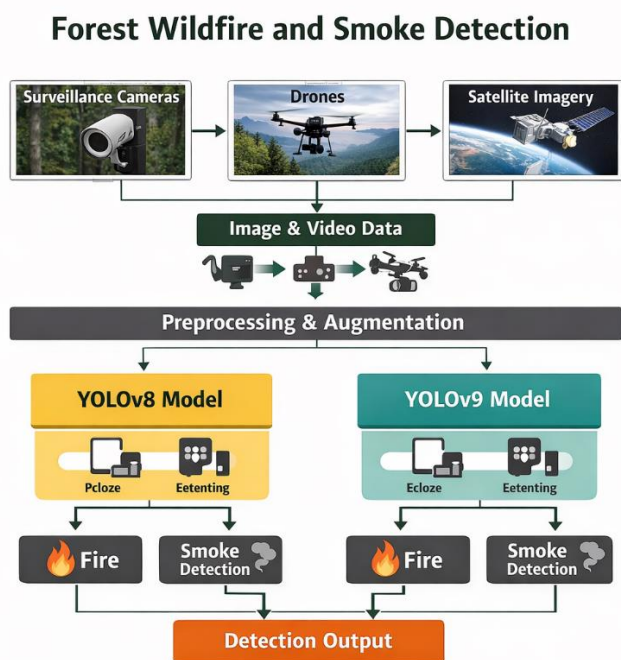


Figure 1 Project Flow

The project flow is shown in **Figure 1**. It begins with data collection, where images and videos are gathered from sources like satellites, drones, and surveillance cameras. These inputs are then preprocessed by resizing, normalizing, and augmenting the data to prepare it for model training.

The YOLOv8 and YOLOv9 models are trained to detect wildfire and smoke from these images. Once trained, the system captures real-time images, which are processed to detect smoke or fire. If either is detected, the system identifies the type and displays the results. If no smoke or fire is found, the process ends.

II. Problem Statement

The growing frequency and intensity of wildfires, driven by climate change, pose significant threats to ecosystems, communities, and infrastructure. In many parts of the world, wildfires are not only a natural disaster but also an environmental crisis with long-term consequences. Traditional wildfire detection methods, such as manual patrols, satellite imagery, and human surveillance, often suffer from delays and inefficiencies, particularly in large, remote, and forested areas. These limitations hinder timely detection and response, leading to increased damage and risk to human lives and wildlife.

Early detection is critical in mitigating the impacts of wildfires. The ability to identify the presence of smoke or fire at the earliest stage can significantly reduce the response time for firefighting teams and evacuation efforts. However, the current systems struggle to provide real-time, high-accuracy detection across vast, often inaccessible, regions. As such, there is a pressing need for an automated and efficient system capable of detecting smoke and fire in real-time, regardless of the environmental conditions or the scale of the area.

This research addresses the challenge of improving wildfire detection by leveraging state-of-the-art deep learning algorithms, specifically the YOLOv9 and YOLOv8 models. The proposed system aims to overcome the limitations of traditional approaches by using image and video data from surveillance cameras, drones, and satellites to automatically detect and classify wildfire smoke and fire in real-time. This system's efficiency in detection could help reduce the time between fire onset and

intervention, ultimately minimizing the destruction caused by wildfires..

III. Literature survey

YOLO (You Only Look Once) is a state-of-the-art object detection model that has revolutionized real-time detection tasks.

YOLO's ability to process images quickly while maintaining high accuracy makes it suitable for detecting wildfires and smoke in large and dynamic environments. YOLOv9 and YOLOv8, the latest versions of the algorithm, have improved on earlier iterations by enhancing [2] detection precision, especially in complex and cluttered scenarios like forests.

Data collection is one of the most crucial steps in training any machine learning model. For wildfire detection, the data must represent various environmental conditions, including different lighting, weather, and geographical terrains[3] . A diverse dataset is essential to ensure that the model generalizes well and can perform in various real-world conditions. Satellite imagery, drone footage, and surveillance camera feeds are commonly used to gather this data, capturing a wide range of wildfire and smoke scenarios.

Before training the models, the collected data undergoes preprocessing steps such as resizing, normalization, and augmentation. These steps ensure that the input images are uniform and suitable for training. Data augmentation, in particular, helps generate more training samples by applying random transformations such as rotations and flips, which prevents the model from overfitting. By enhancing the training dataset, the model's performance is significantly improved, making it more robust when handling diverse fire scenarios [4].

Training deep learning models like YOLOv8 and YOLOv9 requires substantial computational resources. During the training process, the models learn to minimize a loss function, which measures the difference between predicted and actual outputs. The training procedure involves backpropagation and optimization techniques such as stochastic gradient descent (SGD) to adjust the model's parameters. For wildfire detection, the models are trained [5] to accurately identify smoke and fire, optimizing performance across various environmental conditions.

Once the model detects potential wildfire or

smoke, post-processing techniques such as Non-Maximum Suppression (NMS) are applied. NMS is used to eliminate redundant bounding boxes, ensuring that only the most accurate detection remains. This technique improves the efficiency of the system by reducing false positives and focusing on the most likely locations of fire or smoke [6]. Post-processing ensures that the final outputs are precise and actionable for emergency response systems. After training and testing, the YOLO models are deployed in real-time systems for continuous wildfire monitoring. These systems analyze video streams from various sources, such as drones and satellites, to detect smoke and fire as they occur. Upon detection, the system triggers alerts to notify emergency responders, enabling faster action [7]. This integration into real-time monitoring frameworks ensures that the system is highly effective in rapidly identifying and responding to wildfires in large and remote areas.

IV. Proposed methodology

Overview of the Proposed System

The proposed methodology for wildfire and smoke detection relies on state-of-the-art deep learning algorithms—YOLOv8 and YOLOv9. These algorithms are used for real-time object detection, making them highly suitable for wildfire detection systems, which require both speed and accuracy. The system involves several key steps, including data collection, preprocessing, model training, and deployment. Each of these steps contributes to building a robust and efficient detection system that can be deployed for monitoring large and remote forested areas.

Figure 1 provides an overview of the system's workflow. It illustrates the key steps, starting from data collection through to the final detection and alert system.

1. Data Collection and Preprocessing

4.2.1 Data Collection: The first step in the proposed methodology is collecting data from various sources, including satellite imagery, drone footage, and surveillance camera feeds. These data sources are crucial for gathering real-time visual data of wildfire-prone areas under various environmental conditions, such as clear skies, fog, and varying lighting.

4.2.2 Data Preprocessing: The collected data undergoes preprocessing to standardize and enhance its quality for training. Preprocessing steps include:

- **Resizing:** Images are resized to fit the input size required by the YOLO models [8] (e.g., 416x416 or 608x608 pixels).
- **Normalization:** Pixel values are normalized to a range between 0 and 1, ensuring that the model trains effectively and efficiently.
- **Data Augmentation:** Since wildfires are rare events, the dataset is augmented using techniques like flipping, rotation, and brightness adjustment. This increases the variability of the dataset, helping the model generalize better.
- **Labeling:** Bounding boxes are drawn around areas containing smoke or fire, providing labeled data for the model's supervised learning.

YOLOv8 and YOLOv9 Model Architecture

The heart of the proposed system lies in the YOLO (You Only Look Once) architecture. Both **YOLOv8** and **YOLOv9** are used to detect and classify wildfire and smoke. These models are designed for high-speed processing with high accuracy, making them ideal for detecting objects in real-time.

YOLOv8 Architecture

YOLOv8 improves on earlier versions by introducing enhancements to the network structure, enabling more efficient feature extraction and improved detection of smaller objects, such as smoke plumes. The architecture consists of:

- **Backbone:** The backbone extracts essential features from the input image. It uses deep convolutional layers to capture the most significant visual patterns.
- **Neck:** The neck processes these extracted features and prepares them for object detection, helping the model detect objects at multiple scales, which is essential for detecting both large fires and small smoke plumes.
- **Head:** The head performs the final detection task, predicting bounding boxes and class labels for each object detected (fire, smoke, or other).

YOLOv9 Architecture

YOLOv9 builds upon YOLOv8's strengths and introduces further improvements for higher precision in detecting small, occluded objects in complex environments. It includes:

- **Backbone:** YOLOv9 uses a more advanced backbone for feature extraction, leveraging deep

neural networks optimized for better feature mapping.

- **Neck:** The neck in YOLOv9 integrates feature pyramids, which allows the model to handle objects at different sizes more effectively.
- **Head:** The head is designed to predict multiple bounding boxes for each detected object, increasing the model's flexibility in identifying overlapping or closely located wildfires and smoke sources.

Both versions of YOLO use a **Single Shot MultiBox Detector (SSD)** strategy, enabling the simultaneous prediction of multiple bounding boxes in one pass of the image, thus optimizing real-time detection.

Training the YOLO Models

The YOLO models are

trained on the preprocessed dataset using a supervised learning approach. During training, the models are exposed to labeled images where the locations of fires and smoke are marked by bounding boxes. The training process aims to minimize the loss function, which includes:

- **Localization Loss:** Measures the difference between predicted and true bounding box coordinates.
- **Confidence Loss:** Measures how confident the model is about each prediction.
- **Classification Loss:** Measures the accuracy of the predicted class labels (e.g., fire, smoke).

Optimization techniques like **Stochastic Gradient Descent (SGD)** are used to adjust the model's parameters, enabling it to make more accurate predictions over time.

Post-Processing and Detection

Once the models are trained, they can be deployed for real-time monitoring. When new images or video streams are captured, the models perform the following steps:

- **Inference:** The model processes the input data and predicts bounding boxes for areas with potential smoke or fire.
- **Non-Maximum Suppression (NMS):** To reduce redundant predictions, NMS is applied. This technique ensures that only the most accurate bounding boxes are kept, eliminating overlapping predictions of the same object.
- **Classification:** Each bounding box is classified into categories such as "fire" or "smoke," depending on the detected features.

The final output consists of labeled bounding boxes around detected fire or smoke areas.

v. Results and Discussions

YOLOv8

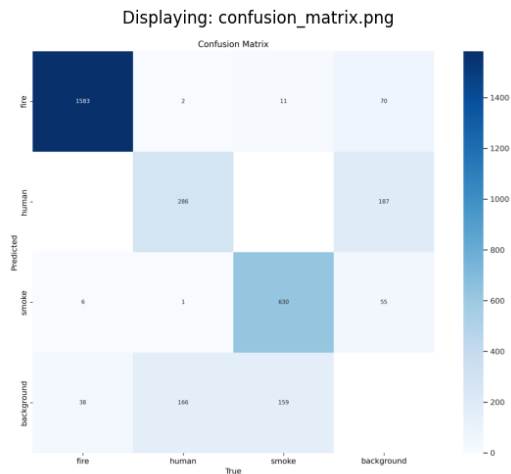


Figure 2 YOLOv8 confusion matrix

The confusion matrix in Figure 2 shows the model's classification results across four categories: fire, human, smoke, and background. The model performed well in detecting fire, with 1583 correct predictions. Smoke detection was also strong, with 630 correct detections. Misclassifications mostly occurred in distinguishing between fire and human, but overall, the model's accuracy was high.

Precision-Confidence Curve (Figure 3)

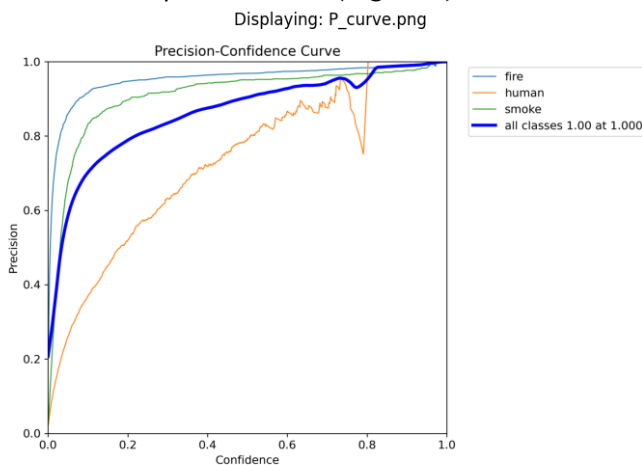


Figure 3 YOLOv8 Precision confidence curve

The precision-confidence curve in Figure 3 illustrates the precision at various confidence levels. For fire and smoke, precision increases sharply as confidence grows, reaching near-perfect values at higher thresholds. Human detection showed gradual precision improvement. The overall precision curve (blue)

confirms the model's strong performance with high precision when confident in its predictions.

YOLOv9

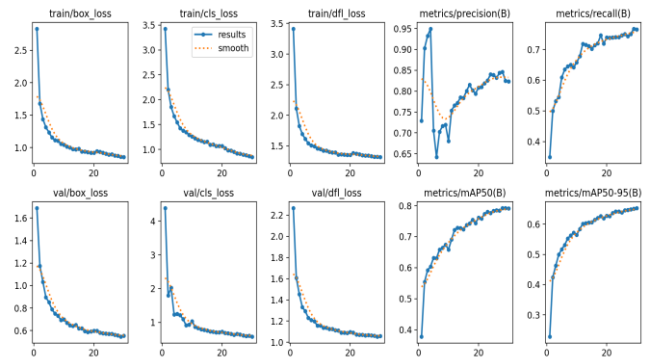


Figure 4 YOLOv9 visualized loss curves, precision, and recall



Figure 5 Test Case Results for YOLOv9

The evaluation of the YOLOv9 model for wildfire and smoke detection was conducted using several performance metrics, including **loss curves**, **precision**, and **recall**, visualized through various figures.

Training and Validation Losses (Figure 4)

The **training and validation loss curves** in **Figure 4** illustrate the model's performance over time. The **training loss** for box, classification (cls), and the **differentiable loss (dfl)** all decreased significantly, indicating that the model was learning and improving during training. The validation losses followed a similar trend, demonstrating the model's ability to generalize effectively to unseen data. The

precision and **recall** metrics show a steady increase, reflecting that the model's ability to accurately detect fire and smoke improved as training progressed.

Precision-Recall (Figure 4)

In **Figure 4**, the **precision** curve exhibits an increase in precision with higher confidence thresholds for both **fire** and **smoke** detections. Similarly, the **recall** curve shows a steady improvement, indicating that the model was able to correctly identify fire and smoke instances as the confidence threshold increased. This shows that the model is efficient in detecting wildfires with minimal false positives when confidence is high.

Test Case Results (Figure 5)

The results of a test case shown in **Figure 5** demonstrate the model's effectiveness in real-time detection. The image displays a **fire** (with 0.98 confidence) and **smoke** (with 0.95 confidence) detected in a real-world scenario. The bounding boxes highlight both objects, confirming the model's ability to detect wildfire-related features in challenging conditions such as night-time and varying visibility. The model's high confidence levels in detecting fire and smoke indicate its robustness for deployment in real-time wildfire monitoring systems.

COMPARISON BETWEEN YOLOv8 AND YOLOv9

Overview of the Algorithms

Both **YOLOv8** and **YOLOv9** are advanced versions of the YOLO (You Only Look Once) object detection model, which is known for its ability to perform real-time detection with high accuracy. These models are applied in detecting fire and smoke in wildfire scenarios, leveraging image data from sources like drones, satellites, and surveillance cameras. YOLOv8 and YOLOv9 both aim to improve object detection in various environmental conditions, but each model has its strengths.

Key Differences and Performance Comparison

- **Accuracy in Fire Detection:** YOLOv8 outperforms YOLOv9 in detecting fire, with **YOLOv8** achieving a **precision** of 0.959 and **recall** of 0.974, significantly higher than YOLOv9. This indicates YOLOv8's effectiveness at identifying fire with high confidence.
- **Smoke Detection:** Both models perform well in detecting smoke, but YOLOv9 provides slightly

better **precision** for smoke (0.921) compared to YOLOv9. However, YOLOv9 still performs robustly with **precision** at 0.82366.

- **Human Detection:** YOLOv9 performs better than YOLOv8 in **human detection**, which is a weak point for YOLOv8 (with **precision** of 0.645 and **recall** of 0.547). YOLOv9's ability to handle diverse detection challenges is a key advantage in real-world applications.
- **Generalization:** YOLOv9 benefits from a more optimized architecture, offering better performance across diverse object categories compared to YOLOv8, particularly for **smaller objects** and **more occlusions**.

For a comprehensive wildfire detection system that needs to detect multiple objects (fire, smoke, and others) in complex, real-time environments, **YOLOv9** is the preferred choice due to its balanced performance across diverse categories.

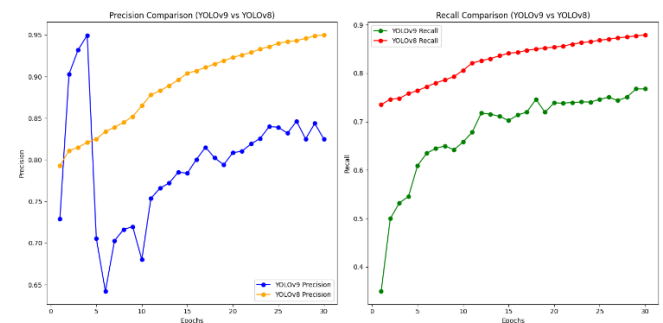


Figure 6 YOLOv9 and YOLOv8 comparison precision and recall

Figure 7 shows a comparison between **YOLOv9** and **YOLOv8** in terms of **precision** (left) and **recall** (right) across 30 epochs.

- **Precision Comparison (left):** The **blue line** represents **YOLOv9's** precision, while the **orange line** shows **YOLOv8's** precision. **YOLOv9** demonstrates more consistent and steady improvement in precision over time, especially after the initial epochs, while **YOLOv8** experiences a more fluctuating performance.
- **Recall Comparison (right):** The **green line** shows **YOLOv9's** recall, and the **red line** shows **YOLOv8's** recall. **YOLOv9** consistently outperforms **YOLOv8** in recall, with a smoother increase and higher final value by the end of the 30 epochs.

This figure highlights the superior consistency and performance of **YOLOv9** in both precision and recall, making it more effective for tasks requiring high detection accuracy, such as wildfire and smoke detection.

VI. Conclusion

This paper presented a deep learning-based

solution for real-time wildfire and smoke detection using YOLOv8 and YOLOv9. Both models demonstrated effective performance, with YOLOv9 providing superior precision and recall compared to YOLOv8, particularly for small and occluded objects like smoke plumes. The proposed system processes visual data from diverse sources such as drones, satellites, and surveillance cameras, offering an efficient and scalable solution for large-scale wildfire monitoring. The results showed that YOLOv9's advanced architecture enhances detection accuracy, enabling quicker response times and more accurate wildfire management. This method has the potential to save lives, protect ecosystems, and reduce the devastating impact of wildfires.

VII. References

- [1] "Climate Change 2021: The Physical Science Basis | Climate Change 2021: The Physical Science Basis." Accessed: Feb. 09, 2026. [Online]. Available: <https://www.ipcc.ch/report/ar6/wg1/>
- [2] Z. Zhang *et al.*, "Robust Semi-supervised Federated Learning for Images Automatic Recognition in Internet of Drones," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5733–5746, Jan. 2022, doi: 10.1109/JIOT.2022.3151945.
- [3] "THREE-DIMENSIONAL ANALYSIS IN ROLLING (METAL FORMING)." Accessed: Feb. 09, 2026. [Online]. Available: https://www.researchgate.net/publication/34956642_THREE-DIMENSIONAL_ANALYSIS_IN_ROLLING_METAL_FORMING
- [4] Y. Zhou, X. Liu, J. Long, Y. Lu, J. Cheng, and J. Li, "Robot path planning based on multi-strategy enhanced aquila optimizer algorithm in complex environments," *Expert Syst. Appl.*, vol. 312, p. 131489, May 2026, doi: 10.1016/J.ESWA.2026.131489.
- [5] Z. Zhang *et al.*, "Robust Semi-supervised Federated Learning for Images Automatic Recognition in Internet of Drones," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5733–5746, Jan. 2022, doi: 10.1109/JIOT.2022.3151945.
- [6] A. Savulescu, B. Siro, C. Ianache, and L. Georgescu, "Simulation of the electric drive of a beam pumping unit and its comparative analysis for different operating frequencies," *2019 6th International Symposium on*

D. Sample Source Code

```
from PIL import Image
import numpy as np
import cv2
import os
import imghdr
from datetime import datetime

app = Flask(__name__)
app.config['SECRET_KEY'] = 'your_secret_key'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

db = SQLAlchemy(app)
bcrypt = Bcrypt(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password = db.Column(db.String(150), nullable=False)
    age = db.Column(db.Integer, nullable=False)
    gender = db.Column(db.String(1), nullable=False)
    mobile = db.Column(db.String(15), nullable=False)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact')
def contact():
    return render_template('contact.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user = User.query.filter_by(email=email).first()
        if user and bcrypt.check_password_hash(user.password, password):
            session['user_id'] = user.id
```

```

        return redirect(url_for('home'))

    else:
        flash('Invalid email or password.', 'danger')
    return render_template('auth.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')
        confirm_password = request.form.get('confirm_password')
        age = request.form.get('age')
        gender = request.form.get('gender')
        mobile = request.form.get('mobile')

        if len(mobile) != 10 or not mobile.isdigit():
            flash('Mobile number must be exactly 10 digits.', 'danger')
            return render_template('auth.html')

        if User.query.filter_by(email=email).first():
            flash('Email address already in use. Please choose a different one.', 'danger')
            return render_template('auth.html')

        if password != confirm_password:
            flash('Passwords do not match.', 'danger')
            return render_template('auth.html')

        if len(password) < 8:
            flash('Password must be at least 8 characters long.', 'danger')
            return render_template('auth.html')

        hashed_password = bcrypt.generate_password_hash(password).decode('utf-8')
        new_user = User(username=username, email=email, password=hashed_password, age=age,
gender=gender, mobile=mobile)
        db.session.add(new_user)
        db.session.commit()

        flash('Registration successful! You can now log in.', 'success')
        return redirect(url_for('login'))
    return render_template('auth.html')

@app.route('/home')
def home():
    return render_template('home.html')

model = YOLO('best.pt')

@app.route('/upload', methods=['GET', 'POST'])

```

```

def upload():
    if request.method == 'POST':
        uploaded_file = request.files['image_file']

        if uploaded_file.filename != "":
            file_path = os.path.join(app.config['UPLOAD_FOLDER'], uploaded_file.filename)
            uploaded_file.save(file_path)
            image = cv2.imread(file_path)
            results = model(image)
            result = results[0]
            boxes = result.bboxes.xywh.cpu().numpy()
            labels = result.names
            target_classes = [
                "fire",
                "human",
                "smoke"
            ]
            detected_classes = [] # List to store detected class names
            for i, box in enumerate(boxes):
                class_id = int(result.bboxes.cls[i])
                label_name = labels[class_id]

                if label_name in target_classes and label_name not in detected_classes:
                    detected_classes.append(label_name) # Add class name to the list if not already present

            x_center, y_center, width, height = box
            x1 = int((x_center - width / 2))
            y1 = int((y_center - height / 2))
            x2 = int((x_center + width / 2))
            y2 = int((y_center + height / 2))

            cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 0), 2)

            label_position = (x1 + 5, y1 + 20)

            (w, h), _ = cv2.getTextSize(label_name, cv2.FONT_HERSHEY_SIMPLEX, 0.7, 2)
            cv2.rectangle(image, (x1, y1), (x1 + w + 10, y1 + h + 10), (0, 255, 0), -1)
            cv2.putText(image, label_name, label_position, cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,
255, 255), 2, cv2.LINE_AA)

            result_image_path = os.path.join(app.config['UPLOAD_FOLDER'], 'result_' +
uploaded_file.filename)
            cv2.imwrite(result_image_path, image)

            # Prepare the message with detected classes
            if detected_classes:
                detection_message = "Classes Detected: " + ", ".join(detected_classes)
            else:
                detection_message = "No Target Class Detected"

            print(detection_message)

```

```
        return render_template('upload.html', result_image=url_for('uploaded_file', filename='result_' +
uploaded_file.filename), detection_message=detection_message)
```

```
    return render_template('upload.html')
```

```
@app.route('/uploads/<filename>')
```

```
def uploaded_file(filename):
```

```
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename)
```

```
@app.route('/logout')
```

```
def logout():
```

```
    session.pop('user_id', None)
```

```
    flash('You have been logged out.', 'info')
```

```
    return redirect(url_for('login'))
```

```
if __name__ == '__main__':
```

```
    with app.app_context():
```

```
        db.create_all()
```

```
    app.run(debug=True)
```

Short Bio-data of the Student

Name of the Student	:	KURUVA RAKSHITHA
Roll Number	:	23102A010433
Date of Birth	:	10-04-2006
Qualification	:	B-Tech
Phone number	:	+91 9701681036
Mail id	:	rakshitha970406@gmail.com
Address	:	kurnool

CO-PO PSO mapping Table

COURSE DESCRIPTION: Identification of topic for the project work; Literature survey; Collection of preliminary data; Identification of implementation tools and methodologies; Performing critical study and analysis of the topic identified; Time and cost analysis; Implementation of the project work; Preparation of thesis and presentation.

COURSE OUTCOMES: After successful completion of this course, the students will be able to:

- CO1.** Create/Design computer science engineering systems or processes to solve complex computer science engineering and allied problems using appropriate tools and techniques following relevant standards, codes, policies, regulations and latest developments.
- CO2.** Consider society, health, safety, environment, sustainability, economics and project management in solving complex computer science engineering and allied problems.
- CO3.** Perform individually or in a team besides communicating effectively in written, oral and graphical forms on computer science engineering systems or processes.

CO-PO-PSO Mapping Table:

Course Outcome	Program Outcomes												Program Outcomes Specific			
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CO1	3	3	3	3	3			3				3				
CO2						3	3				3					
CO3									3	3						

Correlation Level: 3-High; 2-Medium; 1-Low